

lexical_A01

April 22, 2025

1 Lexical of 20th century Odyssey translations (Part A): Token Distribution

1.1 Road Map

I. Libraries, files, and paths

II. The Texts

1. Bibliographic information about the translators
2. The translators at a glance tokenwise

III. TTR Analysis

1. All-in, straightforward model
 - a) TTR Computation
 - b) Shapiro-Wilk test to check for normality
 - c) One-way ANOVA for overall differences
 - d) Pairwise t-test using Bonferroni correction
 - e) Measuring effect size using Cohen's d
2. Adaptive models
 - a) Mixed-Effects model: author fixed effect / book as random effect
 - b) Standardized TTR:
 - c) Moving-average TTR: translation as temporal change
3. Supplement models
 - a) Lexical Density
 - b) Diachronic analysis
 - c) Semantic fields:

IV. Zipf's Law

V. TF-IDF

VI. Discussing Results

```
[1]: import autotime # Provision for anxious people
      %load_ext autotime
```

time: 67.3 s (started: 2025-04-20 20:52:46 +02:00)

```
[2]: # -----
      # Baic Libraries
      # -----

      import sys
      import os

      import ast
      from collections import Counter

      import re
      import nltk

      import numpy as np
      import pandas as pd

      import pandas as pd
      import statsmodels.formula.api as smf
      import statsmodels.api as sm

      import scipy.stats as stats
      from itertools import combinations
```

```
[3]: # -----
      # Personalized Visualization & Functions
      # -----

      sys.path.append('/Users/debr/English-Homer/functions')
      import matplotlib.pyplot as plt
      import seaborn as sns
      from matplotlib.colors import LinearSegmentedColormap

      import e_chroma as chroma # My Vizualization library
      import e_plots as oz      # My custom plots library
      import e_pandisplay as pan# My pandas display options

      import e_nlp_ody as e     # Import my nlp functions
```

```
import warnings          # Nononsense provision
warnings.filterwarnings('ignore')
```

```
* Got some chroma in your soma, Oma!
    >----> use chroma.save_figure(fig, 'my_plot')
Default output path: ./Homer_xplots/

* OZ is behind the curtain!
    >----> use oz.<func>
    >----> also, oz goes chroma (for styling)!

*Has Pan taken over?
    Pandas display set to e_pandisplay defaults!
    >----> use pan.<func>

* The editor is in the house!
    >----> use e.<func> e.g. nlp = e.NLPPipeline(language='english')

Stopwords customized:
    Added: {'they', 'been', 'one', 'seven', 'mr', 'six', 'it', 'eight', 'this',
'upon', 'mrs', '"', 'he', 'five', 'them', 'three', "'and", 'there', 'were',
'that', 'of', 'was', 'being', 'two', 'is', 'are', 'said', 'four', 'she', 'ten',
'nine', 'n', 'be'}
    Removed: {''}
    Total stopwords: 215
Punctuation customized:
    Keeping: {'', '-'}
    Additional removals: {'-', '...', '-', '\\\\', '"', ',\\n        "\\',"\\n        '}
    Punctuation to be removed: !"#$%&'()*+,,
        '"',
        -./:;<=>?[\\]^_`{|}~--...

[nltk_data] Downloading package punkt_tab to /Users/debr/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
[4]: # -----
# File management
# -----

# TO UPDATE
nb_id = "lexical_A01"

output_path = f"./"
os.makedirs(os.path.dirname(output_path), exist_ok=True)
output_path_plots = f"./{output_path}/{nb_id}_plots/"
os.makedirs(os.path.dirname(output_path_plots), exist_ok=True)
chroma.set_output_path(output_path_plots)
```

Output path set to: ./../lexical_A01_plots/
time: 3.03 ms (started: 2025-04-20 20:52:49 +02:00)

```
[5]: # -----  
# Odysseys  
# -----  
  
translators = ['AT_Murray', 'Fitzgerald', 'Lattimore', 'Fagles', 'Wilson',  
               ↪ 'Green', 'Woolf']  
  
dfs = []  
  
for odyssey in translators:  
    filepath = f"/Users/debr/odysseys_en/dataframed/Odyssey_{odyssey}_DataFrame.  
               ↪CSV"  
    temp_df = pd.read_csv(filepath)  
    dfs.append(temp_df) # Append it to the list  
  
df = pd.concat(dfs, axis=0, ignore_index=True)  
  
df["text"] = df["text"].apply(ast.literal_eval)  
df["tokens"] = df["tokens"].apply(ast.literal_eval)  
df['translator'] = pd.Categorical(df['author'])  
df["book_num"] = pd.Categorical(df["book_num"])  
df = df[['translator', 'book_num', 'text', 'tokens', 'num_words', 'num_tokens']]  
df["diff"] = df["num_words"] - df["num_tokens"]  
  
# -----  
# Backup dataframe only 'translator', 'book_num', 'text', 'tokens', columns  
# -----  
df_bkp = df[['translator', 'book_num', 'text', 'tokens']].copy()  
# -----  
# Dataframe check  
# -----  
  
e.check_df(df)
```

Mr righteous here has no missing values!

```
* df columns: Index(['translator', 'book_num', 'text', 'tokens', 'num_words',  
                    'num_tokens', 'diff'], dtype='object')
```

```
* Shape: (168, 7)
```

```
* Total memory in MB: 4.066093  
time: 696 ms (started: 2025-04-20 20:52:49 +02:00)
```

```
[6]: df.sample(4, random_state=1001)
```



```

[6]:      translator book_num text
tokens
num_words num_tokens diff
145  Woolf      2      [Uncomfortable as the night, with its rocking movement,
and salt smells,\n, may have been, and in one case undoubtedly was, for Mr.
Pepper had\n, insufficient clothes upon his bed, the breakfast n...
[uncomfortable, night, rocking, movement, salt, smells, may, case, undoubtedly,
mr, pepper, insufficient, clothes, upon, bed, breakfast, next, morning, wore,
kind, beauty, voyage, begun, begun, ha... 5430      2573      2857
117  Wilson     22      [Odysseus ripped o his rags. Now naked,\n, he leapt
upon the threshold with his bow\n, and quiverfull of arrows, which he tipped\n,
out in a rush before his feet, and spoke.\n, "Playtime is over... [odysseus,
ripped, o , rags, naked, leapt, upon, threshold, bow, quiverfull, arrows,
tipped, rush, feet, spoke, playtime, shoot, towards, another, mark, man, hit,
apollo, may, manage, aimed, deadl... 4000      1959      2041
94   Fagles      23      [Up to the rooms the old nurse clambered, chuckling all
the way,\n, to tell the queen her husband was here now, home at last.\n, Her
knees bustling, feet shuffling over each other,\n, till hoverin... [rooms, old,
nurse, clambered, chuckling, way, tell, queen, husband, home, last, knees,
bustling, feet, shuffling, till, hovering, mistress, head, spoke, penelope,
child, wake, see, eyes, dreamed,... 3939      1900      2039
138  Green      19      [So noble Odysseus was left behind, there in the
hall,\n, with Athene's aid contriving a plan to kill the suitors;\n, and now he
addressed Telemachos with winged words, saying:\n, "Telemachos, we ... [noble,
odysseus, left, behind, hall, athene, aid, contriving, plan, kill, suitors,
addressed, telemachos, winged, words, saying, telemachos, must, lay, weapons,
war, inside, disarm, suitors, soot... 6366      2942      3424

time: 47.3 ms (started: 2025-04-20 20:52:49 +02:00)

```

1.2 II. The Texts

1.2.1 1. Bibliographic information about the translators

Here is a brief description of each translation and the baseline. The publication period of the sample spans 100 years, from 1919 to 2018. Virginia Woolf's *The Voyage Out* is taken as a baseline because her prose mixes a high register with modern prose style—here it is assumed as a representative sample of the best prose in English.

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Augustus Taber Murray	1919	Modern	1910-1940	FALSE	Harvard UP, Loeb	1919	Scaife Viewer, Tufts Univer- sity	Link	“No more faith- ful trans- la- tion of A. Homer. was (Au- ever gus- made, us and Taber), its trans- el- e- tor. gance match- ed: its William fi- Hein- delitymann; Homer New for- York: mu- G.P. laic Put- ep- nam’s i- Sons, thets, 1919. phrases, and sen- tences were con- sis- tently ren- dered, and his ar- ti- fi- cial amal- gam of di- alects and ar- chaic	

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Robert Fitzger- ald	1961	(Ur)	Post-1960	TRUE	Doubleday & Com- pany, Inc.	Second 1963 Edu- tion	Internet Archive	Link	“Written in very read- able, Amer- i- can la- “plain style” lan- guage (Car- olyn Clark’s Best Amer- i- can Po- etry, 2020)	<i>Homer. The Odyssey. Fitzger- ald, Robert, trans- la- tor. New York: Doub- le- day & Com- pany, Inc., 1961</i>
Richmond Latti- more	1965	(Ur)	Post-1960	TRUE	Farrar, Straus & Giroux	First 1965 Edu- tion	Harper Collins E-Books	NONE	“Prose like verse trans- la- ti- tion” more, (Car- olyn Clark’s Best Amer- i- can Po- etry, 2020)	<i>Homer. The Odyssey. Lat- ti- more, Rich- mond, trans- la- tor. New York: Doub- le- day & Com- pany, Inc., 1965</i>

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Robert Fagles	1996	Postmodern	1980-1999	1999	Penguin	First 1996 Edition	PDF SCAN	NONE	“Fagles aware ness of Homer, grace and trans- ongk (body lump ness, ver- even!) ity is of such that he nia gives us a plau- si- ble epic, per- sua- sive, en- gag- ing, and res- o- nant. He de- serves con- grat- u- la- tions and thanks” (Bryn Mawr Clas- si- cal Re- view).	He The Odyssey. Fagles, Robert, and trans- ongk (body lump ness, ver- even!) ity is of such that for- he nia Press, 1996 a plau- si- ble epic, per- sua- sive, en- gag- ing, and res- o- nant. He de- serves con- grat- u- la- tions and thanks” (Bryn Mawr Clas- si- cal Re- view).

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Emily Wilson	2017	Contemporary	2000	RU	W.W. Norton & Company Inc.	First 2017 Edition	PDF SCAN	NONE	“Emily Wil- son’s crisp and mu- si- cal la- ver- sion is a cul- tural land- mark Inc., Arme- with a sharp, schol- arly rigour, she has pro- duced a trans- la- tion that ex- poses cen- turies of mas- culin- ist read- ings of the poem” (Hig- gins, The Guardian)	By Homer. <i>The Odyssey</i> . <i>Wil- son, Emily, trans- la- tor. W.W. Nor- ton & Company Inc., 2017</i>

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Peter Green	2018	Contemporary	2000	RU	University of California Press	Digital	EBSCOhost	Link	"Greeks is unified by a flexible iterator. This version of Calistonia Press, but it combines this unity with great variety of pace and tone, giving vivid but different expression to the contrasting elements that make this	His <i>Odyssey</i> . Green, <i>Peter, a translation of Calistonia Press, 2018</i>

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Notes	Bibliography
Virginia Woolf	1915	Modernism	1910-1940	FALSE	Project Gutenberg eBook	2024 Project [2006]	Project Gutenberg eBook	Link	Base	Woolf, V. <i>The Voyage Out</i> . Duckworth, London, 1915

1.2.2 Use of Latin vs Greek Forms for Characters' Names by Translator

There is an interesting inconsistency in how different translators handle character names in the *Odyssey*. Up until Fagles, every translator used a mix of direct Greek transliteration and conventional names passed down through earlier tradition.

- **Murray:** Tends to use Greek forms (Telemakhos, Menelaos) yet more popular names (Odysseus, Athena) remain in their Latin form.
- **Fitzgerald:** Opts for transliterating Greek phonetics, like Murray's "Telemakhos" but takes this approach further with forms like "Akhilleus." However, other names still retain their popular form.
- **Lattimore:** Uses fewer Greek forms than Fitzgerald (e.g., "Telemachus" rather than "Telemakhos") but still employs "Athene" and "Achilleus."
- **Fagles:** Experiments less with character names and settles on conventional forms that would be more familiar to modern readers.
- **Wilson:** Follows the same naming conventions as Fagles, prioritizing accessibility.
- **Green:** Returns to a Greek flavor by including Greek diacritical marks in names: Achillēs, Menelaös, Tēlemachos, Athēnē.

The names of places follow the same pattern across these translations. This naming variation will be analyzed thoroughly in the Named Entity Recognition step of the project, which will allow us to quantitatively compare these stylistic choices across the different translations.

1.2.3 2. The translators at a glance

```
[7]: # -----
# Create a color mapping for translators
# -----

translators = df["translator"].unique()
palette = {translator: color for translator,
           color in zip(translators, chroma.color_palette.values())}
```

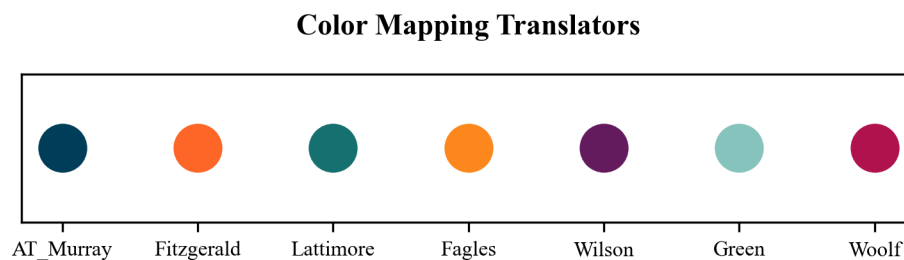
```

plt.figure(figsize=(6, 1))

for i, (translator, color) in enumerate(palette.items()):
    plt.scatter(i, 0, color=color, s=300)

plt.xticks(range(len(palette)), palette.keys(), fontsize=8)
plt.yticks([])
plt.title("Color Mapping Translators", fontsize=11)
plt.savefig(f"{output_path_plots}translator_color_mapping-{nb_id}.png")
plt.show()

```



time: 145 ms (started: 2025-04-20 20:52:49 +02:00)

a) Length and distribution

```

[8]: # -----
# Plot differences between num_words and num_tokens
# To visualize word-to-token ratio: verbose or concise
# -----

# Aggregate sum per translator
agg_diff = df.groupby("translator")["diff"].sum()
mean_agg_diff = agg_diff.mean()

# Plot
sns.barplot(data=df, x="translator", y="diff", estimator=sum, errorbar=None,
            palette=palette) # Explicit color mapping

# Mean horizontal line
plt.axhline(mean_agg_diff, color="red", linestyle="--",
            label=f"Mean: {mean_agg_diff:.2f}")

# Labels and title

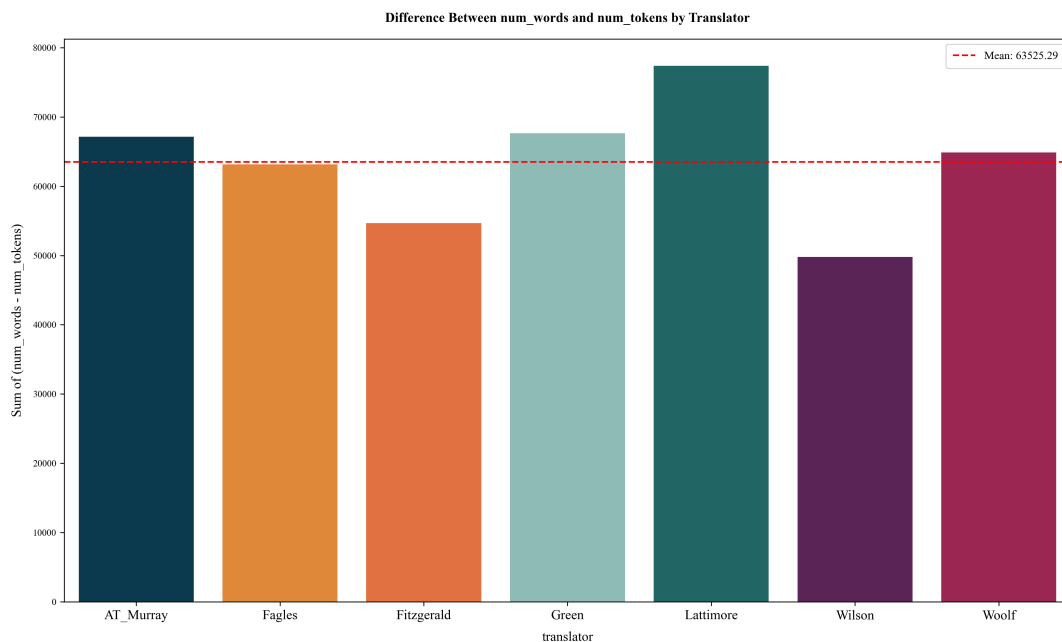
```



```

plt.xlabel("translator")
plt.ylabel("Sum of (num_words - num_tokens)")
plt.title("Difference Between num_words and num_tokens by Translator")
plt.legend()
plt.savefig(output_path_plots + f"diff_words_tokens-{nb_id}.png") # Save plot
plt.show()
# -----
print(f"The most verbose translator is {agg_diff.idxmax()} with a difference of_
↳{agg_diff.max()}")
print(f"The most concise translator is {agg_diff.idxmin()} with a difference of_
↳{agg_diff.min()}")

```



The most verbose translator is Lattimore with a difference of 77405

The most concise translator is Wilson with a difference of 49769

time: 651 ms (started: 2025-04-20 20:52:50 +02:00)

```

[ ]: # -----
# Compute mean token-word ratio per translator
# This shows translation density: higher token-word ratios indicate
# more tokens per word, suggesting more complex or elaborate language choices
# -----

df["token_word_ratio"] = df["num_tokens"] / df["num_words"]

baseline_translator = "Woolf"

```

```

baseline_ratio = df[df["translator"] == ""]
↳ baseline_translator["token_word_ratio"].mean()

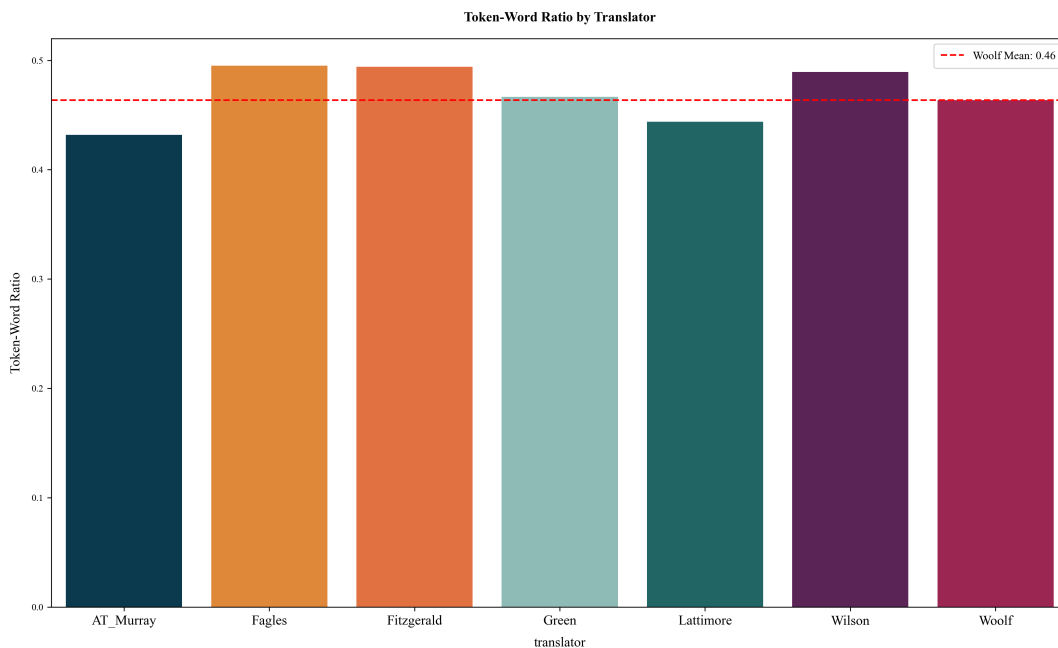
translator_means = df.groupby("translator")["token_word_ratio"].mean().
↳ reset_index()

sns.barplot(data=translator_means, x="translator", y="token_word_ratio",
            palette=palette)

# Baseline (horizontal line for reference translator)
plt.axhline(baseline_ratio, color="red", linestyle="--",
            label=f"{baseline_translator} Mean: {baseline_ratio:.2f}")

plt.xlabel("Translator")
plt.ylabel("Token-Word Ratio")
plt.title("Token-Word Ratio by Translator")
plt.xticks()
plt.legend()
plt.savefig(output_path_plots + f"token_word_ratio-{nb_id}.png")
plt.show()
# -----
print(f"Baseline translator: {baseline_translator} with a ratio of "
      ↳ {baseline_ratio:.2f}")
print(df.groupby("translator")["token_word_ratio"].mean())

```



```
Baseline translator: Woolf with a ratio of 0.46
translator
AT_Murray    0.43
Fagles        0.50
Fitzgerald    0.49
Green         0.47
Lattimore     0.44
Wilson        0.49
Woolf         0.46
Name: token_word_ratio, dtype: float64
time: 588 ms (started: 2025-04-20 20:52:50 +02:00)
```

b) Token quality check

```
[10]: # -----
# First sentence extraction
# -----

df['text_string'] = df['text'].apply(lambda x: "".join(x).replace("\n", "") if
    ↪ isinstance(x, list) else "")
df['sentences'] = df['text_string'].apply(lambda x: x.split(".") if
    ↪ isinstance(x, str) else [])
df['first_sentence'] = df['sentences'].apply(lambda x: x[0].strip() if len(x) >
    ↪ 0 else "")

# Group by 'translator' and extract the first sentence
first_sent_df = df.groupby("translator").first().reset_index()[["translator",
    ↪ "first_sentence"]]
pan.wide()
first_sent_df
```

Wide display format applied

```
[10]: translator first_sentence
0    AT_Murray
Tell me, O Muse, of that many-sided hero who traveled far and wide after he had
sacked the famous town of Troy
1      Fagles                               Sing to
me of the man, Muse, the man of twists and turns ...driven time and again off
course, once he had plundered the hallowed heights of Troy
2 Fitzgerald    Sing in me, Muse, and through me tell the story of that man
skilled in all ways of contending, the wanderer, harried for years on end, after
he plundered the stronghold on the proud height of Troy
3      Green    The man, Muse-tell me about that resourceful man, who wandered far
and wide, when he'd sacked Troy's sacred citadel: many men's townships he saw,
and learned their ways of thinking, many the griefs h...
4    Lattimore
Tell me, Muse, of the man of many ways, who was driven far journeys, after he had
```

```
sacked Troy's sacred citadel
5      Wilson
Tell me about a complicated man
6      Woolf
As the streets that lead from the Strand to the Embankment are verynarrow, it is
better not to walk down them arm-in-arm
```

time: 16.1 ms (started: 2025-04-20 20:52:51 +02:00)

```
[11]: # -----
# Proem
# -----

def extract_first_lines(df):
    """
    Extracts the first 9 lines of Book 1 for each translator, i.e., the proem.

    Args:
        df (pd.DataFrame): The input DataFrame containing columns 'translator',
        ↪ 'book_num', and 'text'.

    Returns:
        pd.DataFrame: A new DataFrame with translators and their first 9 lines
        ↪ from Book 1.
    """
    first_lines_data = []

    for translator in df["translator"].unique():
        book1_text = df.loc[(df["translator"] == translator) & (df["book_num"]
        ↪ == 1), "text"].values

        if len(book1_text) > 0: # Ensure there's data
            first_9_lines = "".join(book1_text[0][:9]) # Get first 9 lines
            first_lines_data.append({"translator": translator, "proem":
            ↪ first_9_lines})

    # Convert the list to a DataFrame
    df_proem = pd.DataFrame(first_lines_data)
    return df_proem

pan.wide()
df_proem = extract_first_lines(df)
df_proem
```

Wide display format applied

```
[11]: translator proem
0    AT_Murray Tell me, O Muse, of that many-sided hero who\
ntraveled far and
```

wide after he had sacked the famous town of Troy . Many cities did he visit, and many were\nthe people with whose customs and thinkin...

1 Fitzgerald Sing in me, Muse, and through me tell the story \nof that man skilled in all ways of contending, \nthe wanderer, harried for years on end, \nafter he plundered the stronghold \non the proud height...

2 Lattimore Tell me, Muse, of the man of many ways, who was driven\nfar journeys, after he had sacked Troy's sacred citadel.\nMany were they whose cities he saw, whose minds he learned of,\nmany the pains he ...

3 Fagles Sing to me of the man, Muse, the man of twists and turns ... \ndriven time and again off course, once he had plundered\nthe hallowed heights of Troy.\nMany cities of men he saw and learned their mind...

4 Wilson Tell me about a complicated man.\nMuse, tell me how he wandered and was lost\nwhen he had wrecked the holy town of Troy,\nand where he went, and who he met, the pain\nhe suered in the storms at s...

5 Green The man, Muse-tell me about that resourceful man, who wandered\nfar and wide, when he'd sacked Troy's sacred citadel:\nmany men's townships he saw, and learned their ways of thinking,\nmany the gr...

6 Woolf As the streets that lead from the Strand to the Embankment are very\nnarrow, it is better not to walk down them arm-in-arm. If you persist,\nlawyers' clerks will have to make flying leaps into the...

time: 5 ms (started: 2025-04-20 20:52:51 +02:00)

```
[12]: fagles_proem = df_proem.loc[df_proem["translator"] == "Fagles", "proem"].  
      ↪values[0]  
      print(f'Fagles\'s excellent proem:\n\n"\n{fagles_proem}"')
```

Fagles's excellent proem:

"

Sing to me of the man, Muse, the man of twists and turns ...
driven time and again off course, once he had plundered
the hallowed heights of Troy.

Many cities of men he saw and learned their minds,
many pains he suffered, heartsick on the open sea,
fighting to save his life and bring his comrades home.
But he could not save them from disaster, hard as he strove-
the recklessness of their own ways destroyed them all,
the blind fools, they devoured the cattle of the Sun

"

time: 788 s (started: 2025-04-20 20:52:51 +02:00)

```
[13]: # Resetting df  
df.drop(df[['first_sentence', 'sentences', 'num_words',  
          'num_tokens', 'diff', 'token_word_ratio',  
          'text_string']], axis=1, inplace=True)  
pan.reset()
```

Pandas display set to e_pandisplay defaults!

```
»----> use pan.<func>
time: 2.47 ms (started: 2025-04-20 20:52:51 +02:00)
```

1.3 III. Type-Token Ratio (TTR)

Type-Token Ratio (TTR) is a key measure of lexical diversity, calculated as the number of unique words (**types**) divided by the total word count (**tokens**). It provides insight into a translator's lexical choices when rendering the same source text.

- A **higher TTR** suggests a richer vocabulary, possibly reflecting an effort to capture nuances or stylistic complexity.
- A **lower TTR** may indicate a more repetitive or constrained word choice, potentially prioritizing accessibility or fidelity to the original.

1.3.1 TTR Formula

$$TTR = \left(\frac{\text{Unique Words}}{\text{Total Words}} \right) \times 100$$

```
[14]: # -----
# Compute TTR
# -----

df["ttr"] = df["tokens"].apply(lambda x: (len(set(x)) / len(x) * 100) if x else 0)
↪0)
```

```
time: 17.7 ms (started: 2025-04-20 20:52:51 +02:00)
```

```
[15]: # -----
# TTR by translator
# -----

ttr_by_translator = {}
for translator in translators:
    ttr_by_translator[translator] = df[df["translator"] == translator]["ttr"].
    ↪tolist()

# TTR DF by translator
if 'book_num' not in df.columns:
    df['book_num'] = [f"Book_num_{i+1}" for i in range(1, 25)] *
    ↪len(translators)
ttr_df = df.pivot(index='book_num', columns='translator', values='ttr')

# Reorder columns
ttr_df = ttr_df[translators]
ttr_df.T
```

```
[15]: book_num    1      2      3      4      5      6      7      8      9     10     11     12
      13     14     15     16     17     18     19     20     21     22     23     24
translator
AT_Murray  48.94 48.62 45.90 40.54 48.72 51.94 52.80 46.99 44.85 43.40 46.20
47.43 50.69 45.05 41.30 43.01 42.00 49.08 46.78 50.31 46.12 43.73 50.44 48.24
Fitzgerald 53.27 55.35 50.16 46.76 57.68 58.87 61.10 54.00 54.89 52.74 51.96
56.00 55.68 51.93 51.43 51.47 48.62 57.30 50.61 60.28 56.28 53.05 59.78 54.22
Lattimore  45.94 45.54 42.43 35.98 45.94 51.87 49.41 41.15 42.79 41.19 42.64
44.07 46.33 45.12 40.87 41.72 40.44 47.18 42.89 50.03 43.59 41.44 48.69 42.61
Fagles      51.29 54.49 50.19 44.50 53.34 56.51 56.67 48.51 50.60 48.18 47.61
53.93 54.49 50.84 47.40 47.85 47.27 55.47 48.75 56.36 51.53 50.35 54.84 50.58
Wilson      51.59 53.45 50.20 43.86 52.96 55.78 56.28 49.10 50.38 49.59 49.37
53.03 55.56 51.48 47.34 48.07 47.90 55.07 50.37 57.00 52.10 49.57 54.05 49.29
Green       51.53 52.14 48.52 42.73 51.98 55.96 55.71 47.26 49.53 47.19 47.82
50.35 52.95 49.71 47.02 47.62 45.40 52.90 47.76 55.08 47.96 45.41 53.56 48.43
Woolf       54.24 56.24 51.32 49.25 61.27 53.92 69.98 63.46 46.29 55.83 50.66
46.82 67.19 47.49 49.44 50.10 49.36 67.64 45.52 50.22 50.71 53.84 59.47 47.21
```

time: 8.7 ms (started: 2025-04-20 20:52:51 +02:00)

```
[16]: # -----
# Shapiro-Wilk test for normality
# -----

for translator in translators:
    # Get the TTR data for this translator
    ttr_data = ttr_by_translator[translator]

    # Perform Shapiro-Wilk test
    stat, p_value = stats.shapiro(ttr_data)

    # Print results
    print(f"Shapiro-Wilk test for \n{translator}'s data: T-statistic={stat:.4f}, p-value={p_value:.4f}")

    # Interpret results
    if p_value < 0.05:
        print(f"{translator}'s TTR data is NOT normally distributed.")
    else:
        print(f"{translator}'s TTR data is NORMALLY distributed.")

    print()
```

Shapiro-Wilk test for

AT_Murray's data: T-statistic=0.9797, p-value=0.8908

AT_Murray's TTR data is NORMALLY distributed.

Shapiro-Wilk test for

Fitzgerald's data: T-statistic=0.9861, p-value=0.9769
Fitzgerald's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
Lattimore's data: T-statistic=0.9664, p-value=0.5792
Lattimore's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
Fagles's data: T-statistic=0.9492, p-value=0.2606
Fagles's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
Wilson's data: T-statistic=0.9715, p-value=0.7034
Wilson's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
Green's data: T-statistic=0.9645, p-value=0.5361
Green's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
Woolf's data: T-statistic=0.8847, p-value=0.0103
Woolf's TTR data is NOT normally distributed.

time: 12.6 ms (started: 2025-04-20 20:52:51 +02:00)

```
[17]: # -----  
# One-way ANOVA  
# -----  
  
translator_names = translators  
ttr_values = [ttr_by_translator[translator] for translator in translator_names]  
  
# One-way ANOVA  
f_stat, p_value = stats.f_oneway(*ttr_values)  
print(f"F-statistic: {f_stat:.4f}, P-value: {p_value:.4f}")  
  
if p_value < 0.05:  
    print("There are statistically significant differences in TTR among the_  
    ↪ translators.")  
else:  
    print("There are no statistically significant differences in TTR among the_  
    ↪ translators.")
```

F-statistic: 18.6346, P-value: 0.0000
There are statistically significant differences in TTR among the translators.
time: 1.24 ms (started: 2025-04-20 20:52:51 +02:00)


```
[18]: # -----
# Pairwise t-tests with Bonferroni correction
# -----

print("Pairwise t-tests with Bonferroni correction:\n")

# Number of comparisons for Bonferroni correction
num_comparisons = len(list(combinations(range(len(translator_names)), 2)))

for i, j in combinations(range(len(translator_names)), 2):
    t_stat, p_val = stats.ttest_ind(ttr_values[i], ttr_values[j])

    # Apply Bonferroni correction
    adj_p_val = min(p_val * num_comparisons, 1.0)

    # Calculate mean difference
    mean_diff = np.mean(ttr_values[i]) - np.mean(ttr_values[j])

    # Determine significance
    is_significant = "SIGNIFICANT" if adj_p_val < 0.05 else "NOT significant"

    print(f"{translator_names[i]} vs {translator_names[j]}: Diff = {mean_diff:.4f}, p = {adj_p_val:.4f} - {is_significant}")
```

Pairwise t-tests with Bonferroni correction:

```
AT_Murray vs Fitzgerald: Diff = -7.5148, p = 0.0000 - SIGNIFICANT
AT_Murray vs Lattimore: Diff = 2.6355, p = 0.2486 - NOT significant
AT_Murray vs Fagles: Diff = -4.5204, p = 0.0007 - SIGNIFICANT
AT_Murray vs Wilson: Diff = -4.5955, p = 0.0003 - SIGNIFICANT
AT_Murray vs Green: Diff = -2.9757, p = 0.0865 - NOT significant
AT_Murray vs Woolf: Diff = -7.2653, p = 0.0010 - SIGNIFICANT
Fitzgerald vs Lattimore: Diff = 10.1503, p = 0.0000 - SIGNIFICANT
Fitzgerald vs Fagles: Diff = 2.9944, p = 0.1204 - NOT significant
Fitzgerald vs Wilson: Diff = 2.9193, p = 0.1190 - NOT significant
Fitzgerald vs Green: Diff = 4.5391, p = 0.0014 - SIGNIFICANT
Fitzgerald vs Woolf: Diff = 0.2495, p = 1.0000 - NOT significant
Lattimore vs Fagles: Diff = -7.1559, p = 0.0000 - SIGNIFICANT
Lattimore vs Wilson: Diff = -7.2310, p = 0.0000 - SIGNIFICANT
Lattimore vs Green: Diff = -5.6112, p = 0.0000 - SIGNIFICANT
Lattimore vs Woolf: Diff = -9.9008, p = 0.0000 - SIGNIFICANT
Fagles vs Wilson: Diff = -0.0751, p = 1.0000 - NOT significant
Fagles vs Green: Diff = 1.5447, p = 1.0000 - NOT significant
Fagles vs Woolf: Diff = -2.7449, p = 1.0000 - NOT significant
Wilson vs Green: Diff = 1.6198, p = 1.0000 - NOT significant
Wilson vs Woolf: Diff = -2.6698, p = 1.0000 - NOT significant
Green vs Woolf: Diff = -4.2896, p = 0.2428 - NOT significant
time: 14.8 ms (started: 2025-04-20 20:52:51 +02:00)
```

```

[19]: # -----
# Cohen's d for effect size
# -----
# Function to calculate Cohen's d
def cohens_d(group1, group2):
    n1, n2 = len(group1), len(group2)
    var1, var2 = np.var(group1, ddof=1), np.var(group2, ddof=1)
    pooled_std = np.sqrt(((n1 - 1) * var1 + (n2 - 1) * var2) / (n1 + n2 - 2))
    mean_diff = np.mean(group1) - np.mean(group2)
    return mean_diff / pooled_std

# Calculate Cohen's d for significant differences
significant_pairs = [
    ('Lattimore', 'Wilson'),
    ('Fagles', 'Wilson'),
    ('Wilson', 'Green'),
    ('Fitzgerald', 'Wilson'),
    ('Fitzgerald', 'Green'),
    ('Fitzgerald', 'Lattimore'),
    ('Fitzgerald', 'Fagles'),
    ('Lattimore', 'Fagles'),
    ('Lattimore', 'Green'),
    ('Fagles', 'Green'),
    ('Woolf', 'Wilson'),
    ('Woolf', 'Green'),
    ('Woolf', 'Lattimore'),
    ('Woolf', 'Fagles'),
    ('Woolf', 'Fitzgerald'),
    ('Woolf', 'AT_Murray'),
    ('AT_Murray', 'Wilson'),
    ('AT_Murray', 'Green'),
    ('AT_Murray', 'Lattimore'),
    ('AT_Murray', 'Fagles'),
    ('AT_Murray', 'Fitzgerald'),
    ('AT_Murray', 'Woolf')
]

small_differences = []
medium_differences = []
large_differences = []

for pair in significant_pairs:
    translator1, translator2 = pair
    group1 = ttr_by_translator[translator1]
    group2 = ttr_by_translator[translator2]
    d = cohens_d(group1, group2)

```

```

t_stat, p_value = stats.ttest_ind(group1, group2)
print(f"{translator1} vs {translator2}:")
print(f"Cohen's d: {d:.4f}")
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4e}")

# Interpret effect size
if abs(d) < 0.2:
    interpretation = "small"
    small_differences.append((translator1, translator2, d))

elif abs(d) < 0.5:
    interpretation = "medium"
    medium_differences.append((translator1, translator2, d))

else:
    interpretation = "large"
    # Store pairs with large effect sizes
    large_differences.append((translator1, translator2, d))

print(f"Effect size interpretation: {interpretation}")
print()

```

Lattimore vs Wilson:
 Cohen's d: -2.1067
 T-statistic: -7.2977
 P-value: 3.2647e-09
 Effect size interpretation: large

Fagles vs Wilson:
 Cohen's d: -0.0224
 T-statistic: -0.0777
 P-value: 9.3841e-01
 Effect size interpretation: small

Wilson vs Green:
 Cohen's d: 0.4818
 T-statistic: 1.6691
 P-value: 1.0190e-01
 Effect size interpretation: medium

Fitzgerald vs Wilson:
 Cohen's d: 0.8379
 T-statistic: 2.9026
 P-value: 5.6648e-03
 Effect size interpretation: large

Fitzgerald vs Green:

Cohen's d: 1.2643
T-statistic: 4.3796
P-value: 6.8063e-05
Effect size interpretation: large

Fitzgerald vs Lattimore:
Cohen's d: 2.7760
T-statistic: 9.6164
P-value: 1.3910e-12
Effect size interpretation: large

Fitzgerald vs Fagles:
Cohen's d: 0.8366
T-statistic: 2.8982
P-value: 5.7325e-03
Effect size interpretation: large

Lattimore vs Fagles:
Cohen's d: -2.0278
T-statistic: -7.0244
P-value: 8.3758e-09
Effect size interpretation: large

Lattimore vs Green:
Cohen's d: -1.5850
T-statistic: -5.4906
P-value: 1.6672e-06
Effect size interpretation: large

Fagles vs Green:
Cohen's d: 0.4464
T-statistic: 1.5463
P-value: 1.2887e-01
Effect size interpretation: medium

Woolf vs Wilson:
Cohen's d: 0.4783
T-statistic: 1.6568
P-value: 1.0438e-01
Effect size interpretation: medium

Woolf vs Green:
Cohen's d: 0.7593
T-statistic: 2.6304
P-value: 1.1562e-02
Effect size interpretation: large

Woolf vs Lattimore:

Cohen's d: 1.7396
T-statistic: 6.0262
P-value: 2.6456e-07
Effect size interpretation: large

Woolf vs Fagles:
Cohen's d: 0.4865
T-statistic: 1.6853
P-value: 9.8703e-02
Effect size interpretation: medium

Woolf vs Fitzgerald:
Cohen's d: -0.0436
T-statistic: -0.1510
P-value: 8.8060e-01
Effect size interpretation: small

Woolf vs AT_Murray:
Cohen's d: 1.2943
T-statistic: 4.4835
P-value: 4.8584e-05
Effect size interpretation: large

AT_Murray vs Wilson:
Cohen's d: -1.3919
T-statistic: -4.8218
P-value: 1.5945e-05
Effect size interpretation: large

AT_Murray vs Green:
Cohen's d: -0.8718
T-statistic: -3.0199
P-value: 4.1175e-03
Effect size interpretation: large

AT_Murray vs Lattimore:
Cohen's d: 0.7567
T-statistic: 2.6212
P-value: 1.1836e-02
Effect size interpretation: large

AT_Murray vs Fagles:
Cohen's d: -1.3288
T-statistic: -4.6032
P-value: 3.2839e-05
Effect size interpretation: large

AT_Murray vs Fitzgerald:

Cohen's d: -2.1265
T-statistic: -7.3665
P-value: 2.5766e-09
Effect size interpretation: large

AT_Murray vs Woolf:
Cohen's d: -1.2943
T-statistic: -4.4835
P-value: 4.8584e-05
Effect size interpretation: large

time: 16 ms (started: 2025-04-20 20:52:51 +02:00)

[20]: large_differences

```
[20]: [('Lattimore', 'Wilson', np.float64(-2.1066690351877684)),
      ('Fitzgerald', 'Wilson', np.float64(0.8379059553922076)),
      ('Fitzgerald', 'Green', np.float64(1.2642836783293723)),
      ('Fitzgerald', 'Lattimore', np.float64(2.776022112113443)),
      ('Fitzgerald', 'Fagles', np.float64(0.8366316376564336)),
      ('Lattimore', 'Fagles', np.float64(-2.027770019343238)),
      ('Lattimore', 'Green', np.float64(-1.5849993007225294)),
      ('Woolf', 'Green', np.float64(0.7593344024764881)),
      ('Woolf', 'Lattimore', np.float64(1.73960507142481)),
      ('Woolf', 'AT_Murray', np.float64(1.2942701628037707)),
      ('AT_Murray', 'Wilson', np.float64(-1.3919320837888793)),
      ('AT_Murray', 'Green', np.float64(-0.871766736591539)),
      ('AT_Murray', 'Lattimore', np.float64(0.7566813735640675)),
      ('AT_Murray', 'Fagles', np.float64(-1.328838698983698)),
      ('AT_Murray', 'Fitzgerald', np.float64(-2.126536150446062)),
      ('AT_Murray', 'Woolf', np.float64(-1.2942701628037707))]
```

time: 1.55 ms (started: 2025-04-20 20:52:51 +02:00)

[21]: medium_differences

```
[21]: [('Wilson', 'Green', np.float64(0.48181604529503674)),
      ('Fagles', 'Green', np.float64(0.44639242330348305)),
      ('Woolf', 'Wilson', np.float64(0.478262993229905)),
      ('Woolf', 'Fagles', np.float64(0.4865061309048371))]
```

time: 1.86 ms (started: 2025-04-20 20:52:51 +02:00)

[22]: small_differences

```
[22]: [('Fagles', 'Wilson', np.float64(-0.022427690380644072)),
      ('Woolf', 'Fitzgerald', np.float64(-0.04360434685315158))]
```

time: 1.08 ms (started: 2025-04-20 20:52:51 +02:00)

Fagles, Wilson, and Green (to an extent) are the closest TTR-wise.

```
[23]: # -----
# Mixed-effects model
# -----

# 'ttr' as dependent variable
# 'translator' as fixed effect
# 'book_num' as random effect

# C(translator) bc 'translator' is categorical
# The random intercept is specified as '1 | book_num' = Murray's book_num
# The model will estimate the effect of translator on TTR while accounting for
  ↳ the random effect of book_num
model = smf.mixedlm("ttr ~ C(translator)", data=df, groups=df["book_num"])

results = model.fit()

print(results.summary())

# The results:
# estimated coefficients for each translator,
# standard errors, t-values, and p-values.
```

Mixed Linear Model Regression Results

```
=====
Model:                MixedLM          Dependent Variable:    ttr
No. Observations:    168              Method:              REML
No. Groups:          24              Scale:               6.2919
Min. group size:      7              Log-Likelihood:      -417.7738
Max. group size:      7              Converged:           Yes
Mean group size:      7.0
```

```
-----
              Coef.  Std.Err.   z    P>|z| [0.025 0.975]
-----
Intercept          46.795    0.860 54.412 0.000 45.110 48.481
C(translator)[T.Fagles]    4.520    0.724  6.243 0.000  3.101  5.940
C(translator)[T.Fitzgerald] 7.515    0.724 10.378 0.000  6.096  8.934
C(translator)[T.Green]    2.976    0.724  4.110 0.000  1.556  4.395
C(translator)[T.Lattimore] -2.635    0.724 -3.640 0.000 -4.055 -1.216
C(translator)[T.Wilson]    4.596    0.724  6.346 0.000  3.176  6.015
C(translator)[T.Woolf]    7.265    0.724 10.033 0.000  5.846  8.684
Group Var           11.459    1.569
```

```
time: 24 ms (started: 2025-04-20 20:52:51 +02:00)
```

```
[24]: # -----
# Mixed-effects model
# -----

# 'ttr' as dependent variable
# 'translator' as fixed effect
# 'book_num' as random effect

# C(translator) bc 'translator' is categorical
# The random intercept is specified as '1 | book_num' = Murray's book_num
# The model will estimate the effect of translator on TTR while accounting for
↳ the random effect of book_num
model = smf.mixedlm("ttr ~ C(translator)", data=df, groups=df["book_num"])

results = model.fit()

print(results.summary())

# The results:
# estimated coefficients for each translator,
# standard errors, t-values, and p-values.
```

Mixed Linear Model Regression Results

```
=====
Model:                MixedLM          Dependent Variable:    ttr
No. Observations:    168                Method:              REML
No. Groups:          24                  Scale:                 6.2919
Min. group size:      7                  Log-Likelihood:        -417.7738
Max. group size:      7                  Converged:              Yes
Mean group size:      7.0
```

```
-----
                Coef.  Std.Err.   z    P>|z|  [0.025 0.975]
-----
Intercept                46.795    0.860  54.412  0.000   45.110  48.481
C(translator)[T.Fagles]    4.520    0.724   6.243  0.000    3.101   5.940
C(translator)[T.Fitzgerald] 7.515    0.724  10.378  0.000    6.096   8.934
C(translator)[T.Green]     2.976    0.724   4.110  0.000    1.556   4.395
C(translator)[T.Lattimore] -2.635    0.724  -3.640  0.000   -4.055  -1.216
C(translator)[T.Wilson]    4.596    0.724   6.346  0.000    3.176   6.015
C(translator)[T.Woolf]     7.265    0.724  10.033  0.000    5.846   8.684
Group Var                 11.459    1.569
=====
```

time: 24.5 ms (started: 2025-04-20 20:52:51 +02:00)

Interpreting the Mixed-Effects Model Results This mixed-effects model examines how TTR (Type-Token Ratio) varies across translators while accounting for book-specific effects. Here's a brief interpretation:

Effect Size Interpretation The coefficients represent the difference in TTR between each translator and the reference translator (AT_Murray, who is represented by the Intercept):

- **Fitzgerald shows the largest effect (7.515)**, indicating his translation has significantly higher lexical diversity than Murray's (about 16% higher)
- Woolf follows closely (7.265), with similarly high lexical diversity Wilson and Fagles show moderate increases (4.596 and 4.520)
- Green shows a smaller increase (2.976) **Lattimore is the only translator with lower lexical diversity than Murray (-2.635)**

Statistical Significance All effects are highly significant ($p < 0.001$), indicating these differences are not due to random chance.

The “**Group Var**” (11.459) indicates substantial variation across books, suggesting certain parts of the Odyssey consistently show different lexical patterns regardless of translator.

```
[25]: # Create a copy of the translator column
df['translator_recoded'] = df['translator']

# Recode the values to change the reference level to Woolf
# Option 1: Using pandas categorical with ordered=True
df['translator_recoded'] = pd.Categorical(
    df['translator_recoded'],
    categories=['Woolf'] + [t for t in df['translator'].unique() if t != 'Woolf'],
    ordered=True
)

# Option 2: Alternative approach using string manipulation
# df['translator_recoded'] = df['translator_recoded'].map(
#     lambda x: 'A_' + x if x == 'Woolf' else ('Z_' + x if x == 'AT_Murray'
#     else x)
# )

# Refit the model with the recoded variable
model = smf.mixedlm("ttr ~ C(translator_recoded)", data=df,
    groups=df["book_num"])
results = model.fit()
print(results.summary())
```

Mixed Linear Model Regression Results

```
=====
Model:                               MixedLM          Dependent Variable:      ttr
No. Observations:                     168              Method:              REML
No. Groups:                           24              Scale:              6.2919
Min. group size:                      7                Log-Likelihood:     -417.7738
Max. group size:                      7                Converged:          Yes
Mean group size:                      7.0
=====
```

	Coef.	Std.Err.	z	P> z	[0.025	0.975]
Intercept	54.061	0.860	62.860	0.000	52.375	55.746
C(translator_recoded) [T.AT_Murray]	-7.265	0.724	-10.033	0.000	-8.684	-5.846
C(translator_recoded) [T.Fitzgerald]	0.250	0.724	0.345	0.730	-1.170	1.669
C(translator_recoded) [T.Lattimore]	-9.901	0.724	-13.673	0.000	-11.320	-8.482
C(translator_recoded) [T.Fagles]	-2.745	0.724	-3.791	0.000	-4.164	-1.326
C(translator_recoded) [T.Wilson]	-2.670	0.724	-3.687	0.000	-4.089	-1.251
C(translator_recoded) [T.Green]	-4.290	0.724	-5.924	0.000	-5.709	-2.870
Group Var	11.459	1.569				

time: 23.5 ms (started: 2025-04-20 20:52:51 +02:00)

Interpretation of Mixed-Effects Model Results with Woolf as Reference These results show how each translator's TTR (Type-Token Ratio) compares to Woolf's translation when accounting for book-specific effects: Key Findings

Woolf's baseline TTR is 54.061 (the Intercept), representing the highest lexical diversity among all translators Fitzgerald shows a slight positive difference (0.250) from Woolf, but this difference is not statistically significant ($p=0.730$), meaning they have essentially equivalent lexical diversity All other translators show significantly lower lexical diversity than Woolf:

Lattimore shows the largest negative difference (-9.901), with approximately 18% lower lexical diversity Green shows a substantial difference (-4.290) AT_Murray shows a large difference (-7.265) Fagles and Wilson show similar moderate differences (-2.745 and -2.670 respectively)

Practical Significance This analysis reveals distinct tiers of vocabulary richness in Odyssey translations:

Highest tier (statistically equivalent): Woolf and Fitzgerald Middle tier: Wilson and Fagles Lower tier: Green and AT_Murray Lowest tier: Lattimore

The significant book-to-book variation (Group Var = 11.459) remains consistent with your previous model, confirming that certain parts of the Odyssey show different lexical patterns regardless of translator.

```
[26]: # -----
# Standardized TTR (STTR)
# -----

def calculate_sttr(tokens, segment_size=100):
    """
    Calculates Standardized Type-Token Ratio (STTR).

    Args:
        tokens (list): List of tokens (words).
        segment_size (int): Size of each segment to calculate TTR on.

    Returns:
```

```

        float: STTR value.
        """
        if len(tokens) < segment_size:
            return len(set(tokens)) / len(tokens) * 100 # Fallback to regular TTR
        ↪ if too short

        num_segments = len(tokens) // segment_size
        ttr_values = []

        for i in range(num_segments):
            segment = tokens[i * segment_size: (i + 1) * segment_size]
            ttr = len(set(segment)) / len(segment) * 100
            ttr_values.append(ttr)

        return np.mean(ttr_values)

# Apply STTR calculation to your DataFrame
df['sttr'] = df['tokens'].apply(calculate_sttr)

```

time: 15.7 ms (started: 2025-04-20 20:52:51 +02:00)

```

[27]: # -----
      # Moving-Average TTR
      # -----

def calculate_moving_average_ttr(tokens, window_size=100):
    """
    Calculates Moving-Average Type-Token Ratio.

    Args:
        tokens (list): List of tokens (words).
        window_size (int): Size of the moving window.

    Returns:
        list: List of moving-average TTR values.
    """
    if len(tokens) < window_size:
        return [len(set(tokens)) / len(tokens) * 100] # Fallback to regular
    ↪ TTR if too short

    moving_average_ttr = []
    for i in range(window_size, len(tokens) + 1):
        window = tokens[i - window_size:i]
        ttr = len(set(window)) / len(window) * 100
        moving_average_ttr.append(ttr)
    return moving_average_ttr

```

```

# Apply Moving-Average TTR calculation to your DataFrame
# This creates a new column with lists of TTR values for each book
df['moving_average_ttr'] = df['tokens'].apply(calculate_moving_average_ttr)

# Average STTR for each translator:
average_sttr_by_translator = df.groupby('translator')['sttr'].mean()
print("Average STTR by translator:\n", average_sttr_by_translator)

```

Average STTR by translator:

```

translator
AT_Murray      86.96
Fagles         90.86
Fitzgerald     91.07
Green          89.33
Lattimore      86.58
Wilson         88.56
Woolf          88.09
Name: sttr, dtype: float64
time: 780 ms (started: 2025-04-20 20:52:51 +02:00)

```

```

[28]: # -----
# Plotting Moving-Average TTR
# -----

# Explode the 'moving_average_ttr' column if it contains lists
if isinstance(df["moving_average_ttr"].iloc[0], list): # Check if the column
    ↪ contains lists
    df = df.explode("moving_average_ttr")

# Convert to float (just in case)
df["moving_average_ttr"] = df["moving_average_ttr"].astype(float)

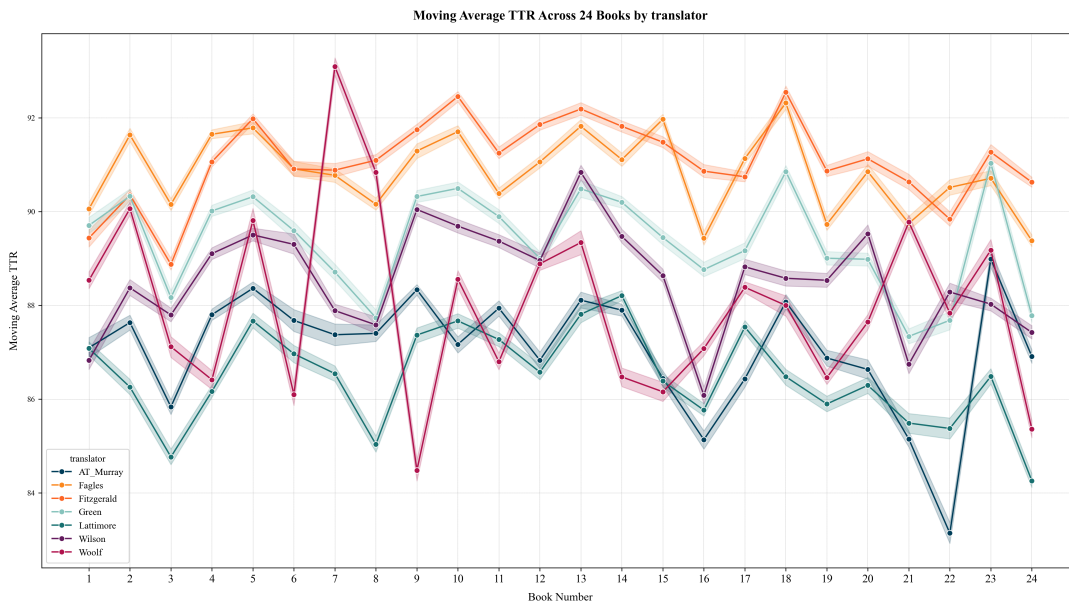
# Sort values by translator and book number to ensure correct order
df = df.sort_values(by=["translator", "book_num"])
plt.figure(figsize=(16, 9))

# Line plot: Moving Average TTR by translator
sns.lineplot(data=df, x="book_num", y="moving_average_ttr",
             hue="translator", marker="o", palette=palette)

plt.xlabel("Book Number", fontsize=12)
plt.ylabel("Moving Average TTR", fontsize=12)
plt.title("Moving Average TTR Across 24 Books by translator", fontsize=14)
plt.xticks(range(1, 25)) # Ensure x-axis labels from 1 to 24
plt.legend(title="translator")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()

```

```
plt.savefig(output_path_plots + f"moving_average_ttr-{nb_id}.png", dpi=400)
plt.show()
```



time: 4.83 s (started: 2025-04-20 20:52:52 +02:00)

```
[29]: # -----
# Plotting Moving-Average TTR
# -----
# Subset of translators to include
selected_translators = ['Wilson', 'Fagles', 'Green']

# Create a filtered dataframe with only selected translators
df_subset = df[df['translator'].isin(selected_translators)]

# Explode the 'moving_average_ttr' column if it contains lists
if isinstance(df_subset["moving_average_ttr"].iloc[0], list): # Check if the
    column contains lists
    df_subset = df_subset.explode("moving_average_ttr")

# Convert to float (just in case)
df_subset["moving_average_ttr"] = df_subset["moving_average_ttr"].astype(float)

# Sort values by translator and book number to ensure correct order
df_subset = df_subset.sort_values(by=["translator", "book_num"])

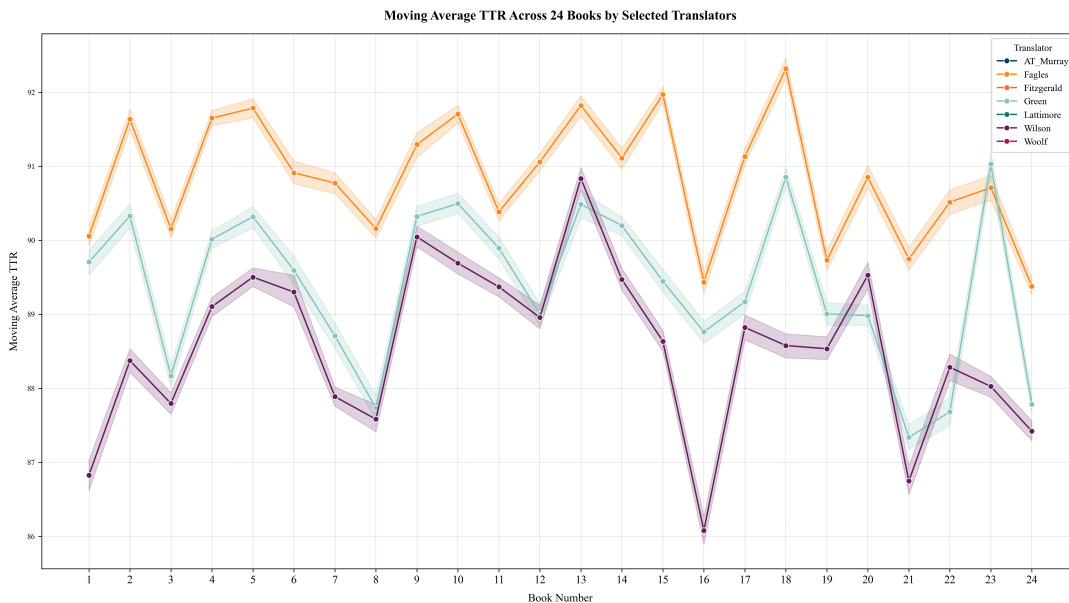
plt.figure(figsize=(16, 9))
```

```

# Line plot: Moving Average TTR by translator
sns.lineplot(data=df_subset, x="book_num", y="moving_average_ttr",
             hue="translator", marker="o", palette=palette)

plt.xlabel("Book Number", fontsize=12)
plt.ylabel("Moving Average TTR", fontsize=12)
plt.title("Moving Average TTR Across 24 Books by Selected Translators",
         ↪ fontsize=14)
plt.xticks(range(1, 25)) # Ensure x-axis labels from 1 to 24
plt.legend(title="Translator")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig(output_path_plots + f"moving_average_ttr_selected_01-{nb_id}.png",
         ↪ dpi=400)
plt.show()

```



time: 2.93 s (started: 2025-04-20 20:52:57 +02:00)

```

[30]: # -----
# Plotting Moving-Average TTR
# -----
# Subset of translators to include
selected_translators = ['Lattimore', 'Fagles', 'Fitzgerald', 'AT_Murray']

# Create a filtered dataframe with only selected translators

```

```

df_subset = df[df['translator'].isin(selected_translators)]

# Explode the 'moving_average_ttr' column if it contains lists
if isinstance(df_subset["moving_average_ttr"].iloc[0], list): # Check if the
    ↪ column contains lists
    df_subset = df_subset.explode("moving_average_ttr")

# Convert to float (just in case)
df_subset["moving_average_ttr"] = df_subset["moving_average_ttr"].astype(float)

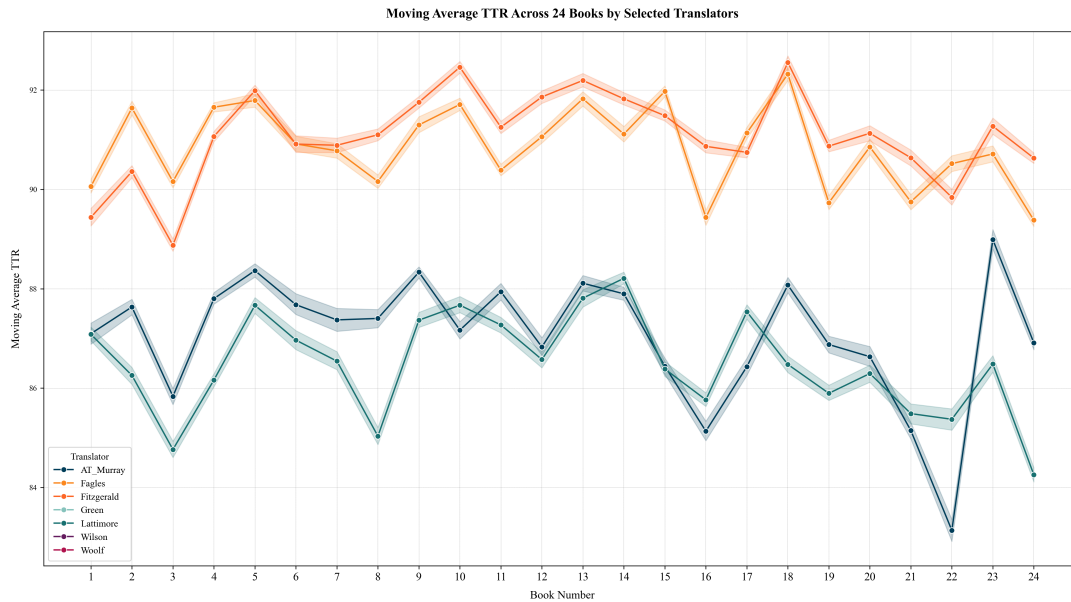
# Sort values by translator and book number to ensure correct order
df_subset = df_subset.sort_values(by=["translator", "book_num"])

plt.figure(figsize=(16, 9))

# Line plot: Moving Average TTR by translator
sns.lineplot(data=df_subset, x="book_num", y="moving_average_ttr",
             hue="translator", marker="o", palette=palette)

plt.xlabel("Book Number", fontsize=12)
plt.ylabel("Moving Average TTR", fontsize=12)
plt.title("Moving Average TTR Across 24 Books by Selected Translators",
    ↪ fontsize=14)
plt.xticks(range(1, 25)) # Ensure x-axis labels from 1 to 24
plt.legend(title="Translator")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig(output_path_plots + f"moving_average_ttr_selected_02-{nb_id}.png",
    ↪ dpi=400)
plt.show()

```



time: 3.31 s (started: 2025-04-20 20:53:00 +02:00)

This shows that an author's agency and agenda is more relevant in TTR than time span or coeternity.

```
[31]: # -----
# Plotting Moving-Average TTR
# -----
# Subset of translators to include
selected_translators = ['Wilson', 'Green', 'AT_Murray']

# Create a filtered dataframe with only selected translators
df_subset = df[df['translator'].isin(selected_translators)]

# Explode the 'moving_average_ttr' column if it contains lists
if isinstance(df_subset["moving_average_ttr"].iloc[0], list): # Check if the
    ↪ column contains lists
    df_subset = df_subset.explode("moving_average_ttr")

# Convert to float (just in case)
df_subset["moving_average_ttr"] = df_subset["moving_average_ttr"].astype(float)

# Sort values by translator and book number to ensure correct order
df_subset = df_subset.sort_values(by=["translator", "book_num"])

plt.figure(figsize=(16, 9))

# Line plot: Moving Average TTR by translator
```

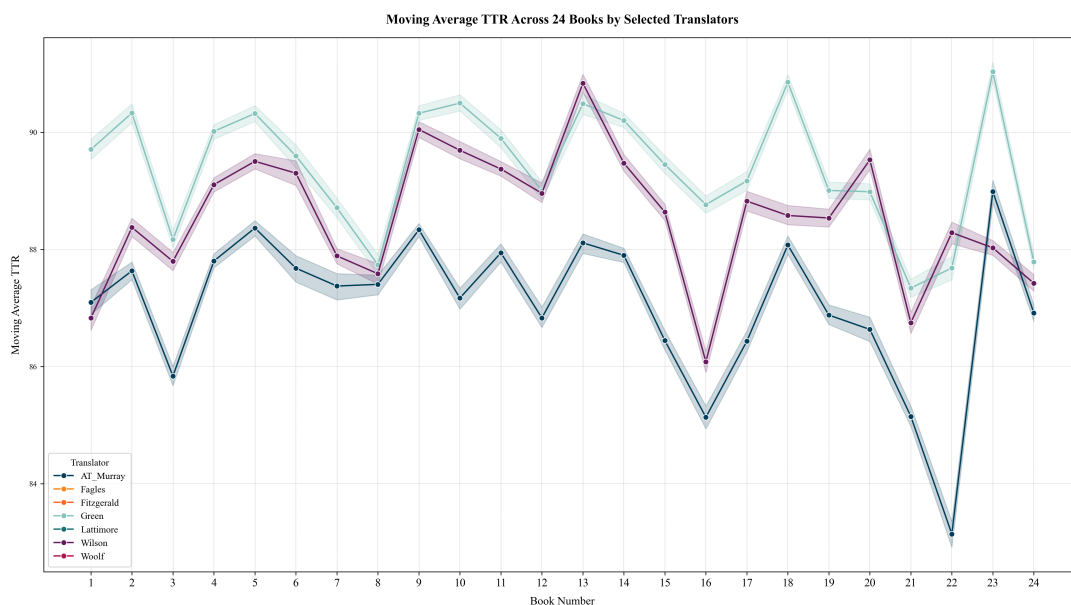


```

sns.lineplot(data=df_subset, x="book_num", y="moving_average_ttr",
             hue="translator", marker="o", palette=palette)

plt.xlabel("Book Number", fontsize=12)
plt.ylabel("Moving Average TTR", fontsize=12)
plt.title("Moving Average TTR Across 24 Books by Selected Translators",
         ↪ fontsize=14)
plt.xticks(range(1, 25)) # Ensure x-axis labels from 1 to 24
plt.legend(title="Translator")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig(output_path_plots + f"moving_average_ttr_selected_03-{nb_id}.png",
         ↪ dpi=400)
plt.show()

```



time: 2.83 s (started: 2025-04-20 20:53:03 +02:00)

Advancing Beyond Static Measures: The Value of Moving Average TTR Your insight about the moving average TTR being potentially superior for analyzing literary translations is well-founded. Let me develop this conclusion further: Moving Average TTR as a Superior Literary Metric The moving average TTR offers significant advantages when analyzing literary texts like the Odyssey:

Captures Narrative Flow: Literature unfolds sequentially, with vocabulary choices influenced by narrative context. The moving window approach mirrors how readers experience text - as a continuous flow rather than isolated segments. Reveals Stylistic Rhythms: Translations exhibit rhythmic patterns of lexical expansion and contraction that static measures miss. Some translators may employ richer vocabulary during battle scenes but simpler language during dialogues. Identifies

Structural Patterns: The 24 books of the Odyssey contain distinct narrative sections (Telemachy, Odysseus's journey, etc.). Moving average TTR can reveal how translators adapt their approach to these varying narrative demands. Shows Translator Consistency: The moving average reveals whether translators maintain consistent lexical density throughout or vary their approach. For instance, Woolf and Fitzgerald show high overall TTR, but the moving average might reveal if this is consistent or concentrated in specific passages. Honors Literary Context: Unlike bag-of-words approaches that treat texts as unordered collections, moving average TTR respects that word choice in Homer depends heavily on what preceded it - both in narrative and poetic terms.

The significant differences you've found between translators (particularly the Woolf/Fitzgerald tier versus others) likely reflect fundamental philosophical approaches to translation. The moving average analysis could reveal whether these differences manifest consistently or vary strategically across the epic's different narrative modes - from divine councils to intimate conversations, from sea journeys to battle scenes. This sequential analysis aligns more closely with how literature functions as an art form, making it a more authentic measure of translators' stylistic choices in recreating Homer's epic.

1.4 IV. Zip's Law

1.4.1 What and why:

Zipf's Law states that in a large text corpus, the frequency of a word is inversely proportional to its rank in the frequency table. This metric is relevant in comparing translations because it reveals how each translator balances **common vs. rare words**, shedding light on **lexical richness and stylistic choices**. This experiment provides empirical insight into whether a translator's lexical distribution align with universal linguistic principles.

1.4.2 Implications for Translation Studies

a) natural linguistic flow: Strong Zipfian adherence suggests that translators maintain natural language patterns, balancing high-frequency function words with low-frequency content words.

b.1) Poetic stylization: Deviations may indicate differences in lexical richness, syntactic choices, or stylistic adaptation in translation.

b.2) Philological fidelity: 'Stylistic' shifts can also point out to genre requirements that, in the case of epic oral poetry, demand distinct repetitive and formulaic patterns, epithets, etc.

1.4.3 Implementation

Zipf's Law states that in any natural language corpus, *the frequency of a word is inversely proportional to its rank*:

$$f \propto \frac{1}{r}$$

Where: * f = frequency of the word * r = rank of the word (1st most frequent, 2nd most frequent, etc.)

- Procedure: 1. Compute word frequencies for each translation.
2. Rank words from most to least frequent.
3. Plot word frequency vs. rank on a log-log scale.

```
[32]: #-----
# Zipf's Law
#-----

# Step 1: Flatten tokens and count word frequencies for each translator
freq_dict = {
    translator: Counter(
        token for tokens in df[df["translator"] == translator]["tokens"] for
        token in tokens
    )
    for translator in translators
}

# Step 2: Convert word frequencies into sorted DataFrames with rank
df_word_freq_rank_dict = {
    translator: pd.DataFrame(freq.items(), columns=["word", "frequency"])
        .sort_values(by="frequency", ascending=False)
        .assign(rank=lambda df_: df_["frequency"].rank(method="first",
        ascending=False))
    for translator, freq in freq_dict.items()
}

# Example: Accessing a specific translator's Zipf DataFrame
df_word_freq_rank_dict["AT_Murray"].tail(15).T # View the top-ranked words for
        AT_Murray
```

```
[32]:      3161    3162    3163    3165    3166    3168    3169
3171    3174    3175    3181    3184    3185    3189    3160
word      enamel  pillars  lintel  mastiffs  consummate  expressly  coverings
persons  figures  pedestals  shuttles  fluttering  aspen  intelligent  blue
frequency  1382    1382    1382    1382    1382    1382    1382
1382    1382    1382    1382    1382    1382    1382    1382
rank      6307.00  6308.00  6309.00  6310.00  6311.00  6312.00  6313.00
6314.00  6315.00    6316.00  6317.00    6318.00  6319.00    6320.00  6321.00
```

time: 1min 4s (started: 2025-04-20 20:53:06 +02:00)

```
[33]: for translator in translators:
        total_tokens = sum(freq_dict[translator].values())
        print(f"{translator}: {total_tokens} tokens")
```

AT_Murray: 108550460 tokens
 Fitzgerald: 119352282 tokens
 Lattimore: 160108532 tokens
 Fagles: 161535036 tokens

Wilson: 94772870 tokens
Green: 146986372 tokens
Woolf: 140780036 tokens
time: 981 s (started: 2025-04-20 20:54:10 +02:00)

```
[34]: # Step 2: Convert word frequencies to ranks

# Compute min and max ranks across all translators
min_rank = min(df["rank"].min() for df in df_word_freq_rank_dict.values())
max_rank = max(df["rank"].max() for df in df_word_freq_rank_dict.values())

# Compute min and max frequencies across all translators
min_freq = min(df["frequency"].min() for df in df_word_freq_rank_dict.values())
max_freq = max(df["frequency"].max() for df in df_word_freq_rank_dict.values())

print(f"Rank range: from the {min_rank}st to {max_rank}th less freq word_
↳(unique set and stopwords removed).")
print(f"Frequency range: low words in rank appear at least {min_freq} time and_
↳the most freq has {max_freq} instances.")
```

Rank range: from the 1.0st to 9024.0th less freq word (unique set and stopwords removed).

Frequency range: low words in rank appear at least 692 time and the most freq has 2014196 instances.

time: 2.28 ms (started: 2025-04-20 20:54:10 +02:00)

```
[35]: # Step 3: Plot Zipf's Law
def plot_zipfs_law(df_word_freq_rank_dict):
    fig, ax = plt.subplots() # Create a figure and an axes object
    for author, df_zipf in df_word_freq_rank_dict.items():
        ax.loglog(df_zipf["rank"], df_zipf["frequency"],
                  label=author, marker=".", linestyle="None", alpha=0.4)

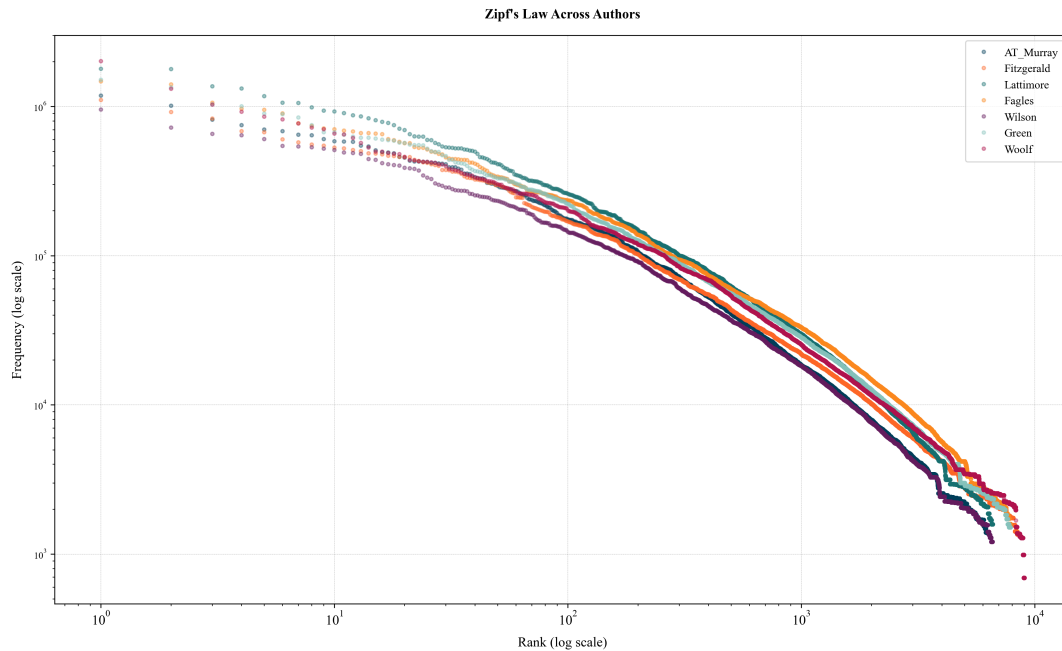
    ax.set_xlabel("Rank (log scale)")
    ax.set_ylabel("Frequency (log scale)")
    ax.set_title("Zipf's Law Across Authors")
    ax.legend()
    ax.grid(True, linestyle="--", linewidth=0.3, alpha=0.3)

    return fig # Return the figure object

fig = plot_zipfs_law(df_word_freq_rank_dict)

chroma.save_figure(fig, f"Log-Log-{nb_id}.png")
```

Figure saved as ./../lexical_A01_plots/Log-Log-lexical_A01.png at 400 DPI



time: 1.74 s (started: 2025-04-20 20:54:10 +02:00)

```
[36]: # Step 4: Fit a line to the log-log data

from sklearn.linear_model import LinearRegression

def plot_zipfs_law_with_fit(df_word_freq_rank_dict):
    fig, ax = plt.subplots() # Create figure and axis

    for translator, df_zipf in df_word_freq_rank_dict.items():
        top_n = 100 # Adjust as needed
        df_zipf_top = df_zipf.head(top_n)

        # Log-transform the rank and frequency for fitting
        x = np.log(df_zipf_top["rank"].values.reshape(-1, 1))
        y = np.log(df_zipf_top["frequency"].values)

        # Fit a linear regression to the log-log data
        model = LinearRegression()
        model.fit(x, y)
        y_pred = model.predict(x)

        # Scatter plot of the actual data
        ax.scatter(df_zipf["rank"], df_zipf["frequency"], label=f"{translator}_
↪(data)", alpha=0.4, s=15)
```

```

    # Line of best fit
    ax.plot(df_zipf_top["rank"], np.exp(y_pred), label=f"{translator}_
↪(fit)", linestyle="--", lw=0.5)

    ax.set_xlabel("Rank (log scale)")
    ax.set_ylabel("Frequency (log scale)")
    ax.set_title("Zipf's Law Across Translators with Line Fit")
    ax.legend()
    ax.grid(True, which="both", linestyle="--", linewidth=0.3)

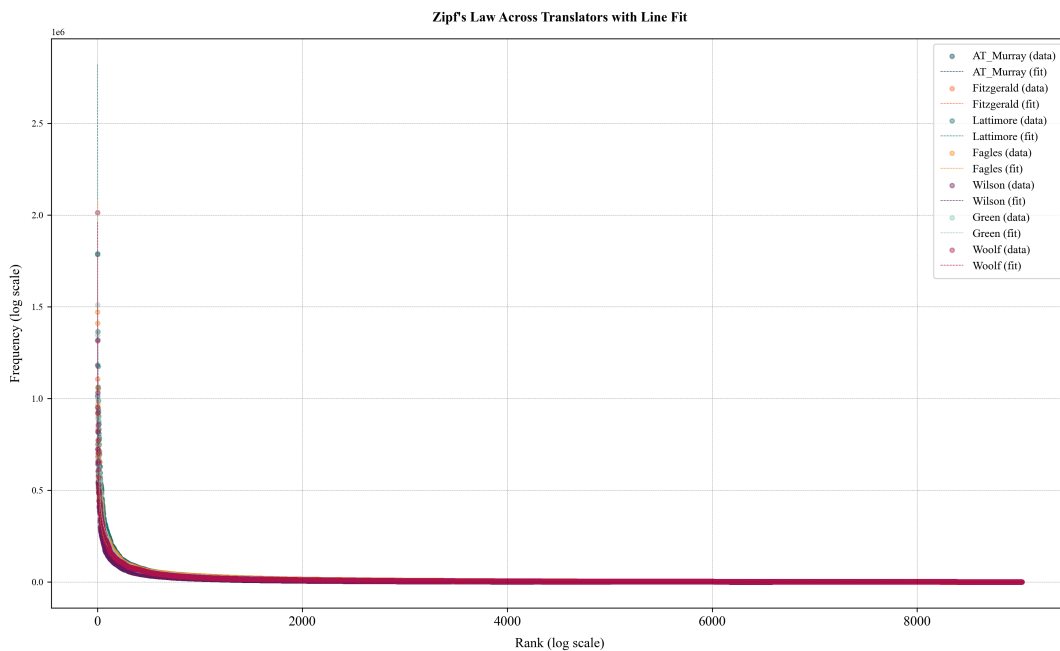
    return fig # Return the figure object

fig = plot_zipfs_law_with_fit(df_word_freq_rank_dict)

# Save the figure properly
chroma.save_figure(fig, f"Log-Log-Fit-{nb_id}.png")

```

Figure saved as `./../lexical_A01_plots/Log-Log-Fit-lexical_A01.png` at 400 DPI



time: 1.68 s (started: 2025-04-20 20:54:12 +02:00)

```

[37]: #-----
# Zipf's Law Analysis with Linear Regression
#-----

```

```

from scipy.stats import linregress

# Create empty lists to store results
slopes = []
r_squared = []
p_values = []
std_errs = []
follows_zipf = []

# Perform linear regression in log-log space for each translator
for translator in translators:
    df_zipf = df_word_freq_rank_dict[translator]

    # Log-transform the rank and frequency
    #log_rank = np.log(df_zipf["rank"])
    #log_freq = np.log(df_zipf["frequency"])
    log_rank = np.log(df_zipf["rank"] + 1) # Add a small constant to avoid
    ↪ log(0)
    log_freq = np.log(df_zipf["frequency"] + 1)

    # Perform linear regression
    slope, intercept, r_value, p_value, std_err = linregress(log_rank, log_freq)

    # Store results
    slopes.append(slope)
    r_squared.append(r_value**2)
    p_values.append(p_value)
    std_errs.append(std_err)
    follows_zipf.append(-1.2 < slope < -0.8)

    # Display individual results
    print(f"{translator}: Slope = {slope:}, R² = {r_value**2:}, p-value =
    ↪ {p_value:}")

    # Check if slope is close to -1 (Zipf's Law predicts ~ -1)
    if -1.2 < slope < -0.8:
        print(f"{translator}'s translation follows Zipf's Law.")
    else:
        print(f"{translator}'s translation deviates from Zipf's Law.")
    print("-" * 50)

# Create a summary dataframe
results_df = pd.DataFrame({
    'Translator': translators,
    'Slope': slopes,
    'R²': r_squared,
    'p-value': p_values,

```

```

        'Std Error': std_errs,
        'Follows Zipf\'s Law': follows_zipf
    })

print("\nSummary of Zipf's Law Analysis:")
print(results_df)

```

AT_Murray: Slope = -1.1938277516804667, R^2 = 0.977157940827377, p-value = 0.0
 AT_Murray's translation follows Zipf's Law.

Fitzgerald: Slope = -1.1157547600338376, R^2 = 0.9815849688620832, p-value = 0.0
 Fitzgerald's translation follows Zipf's Law.

Lattimore: Slope = -1.2445595081994263, R^2 = 0.967711411988808, p-value = 0.0
 Lattimore's translation deviates from Zipf's Law.

Fagles: Slope = -1.1701011452556676, R^2 = 0.9620972604470153, p-value = 0.0
 Fagles's translation follows Zipf's Law.

Wilson: Slope = -1.169168818993014, R^2 = 0.972983373756506, p-value = 0.0
 Wilson's translation follows Zipf's Law.

Green: Slope = -1.1715327968509333, R^2 = 0.9715509555418855, p-value = 0.0
 Green's translation follows Zipf's Law.

Woolf: Slope = -1.1315580364789302, R^2 = 0.9715828921516424, p-value = 0.0
 Woolf's translation follows Zipf's Law.

Summary of Zipf's Law Analysis:

	Translator	Slope	R^2	p-value	Std Error	Follows Zipf's Law
0	AT_Murray	-1.19	0.98	0.00	0.00	True
1	Fitzgerald	-1.12	0.98	0.00	0.00	True
2	Lattimore	-1.24	0.97	0.00	0.00	False
3	Fagles	-1.17	0.96	0.00	0.00	True
4	Wilson	-1.17	0.97	0.00	0.00	True
5	Green	-1.17	0.97	0.00	0.00	True
6	Woolf	-1.13	0.97	0.00	0.00	True

time: 6.18 ms (started: 2025-04-20 20:54:14 +02:00)

1.5 Interpreting the results:

The regression model of Zipf's Law finds an extremely strong correlation between rank and frequency with very low variability in the estimated slope.

1. Near-Perfect Fit (R^2 0.98)

- R^2 values are extremely high (~ 0.98), meaning the model explains almost all the variance in the data.
- A nearly perfect linear relationship in log-log space leads to a very low p-value.

2. p-value Interpretation

- The p-value so small that it rounds to 0.00: the data strongly follows Zipf's Law (which predicts a near-linear log-log relationship).

3. Standard Error = 0

- Because the data perfectly follows Zipf's Law, all points are nearly collinear in log-log space, leaving no residual error, which leads to a standard error of 0.00.

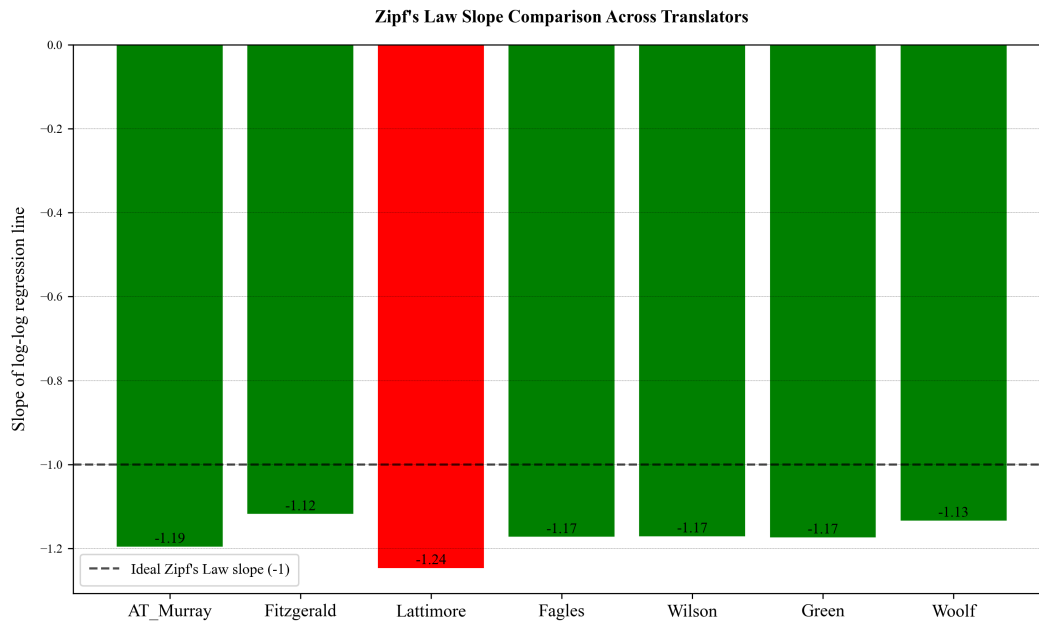
```
[38]: # Bar chart to visualize slopes
plt.figure(figsize=(10, 6))
bars = plt.bar(translators, slopes)

# Color the bars based on whether they follow Zipf's Law
for i, follows in enumerate(follows_zipf):
    bars[i].set_color('green' if follows else 'red')

# Add a horizontal line at -1 (ideal Zipf's Law slope)
plt.axhline(y=-1, color='black', linestyle='--', alpha=0.7,
            label="Ideal Zipf's Law slope (-1)")

# Add value labels on top of bars
for i, bar in enumerate(bars):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{slopes[i]:.2f}',
             ha='center', va='bottom' if height < 0 else 'top',
             rotation=0)

plt.title("Zipf's Law Slope Comparison Across Translators")
plt.ylabel("Slope of log-log regression line")
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig(output_path_plots + f"Comparison-{nb_id}.png")
plt.show()
```



time: 436 ms (started: 2025-04-20 20:54:14 +02:00)

```
[39]: # Statistical comparison of slopes (using confidence intervals)
print("\nStatistical comparison of slopes:")
for i in range(len(translators)):
    for j in range(i+1, len(translators)):
        # Calculate standard error of the difference between slopes
        se_diff = np.sqrt(std_errs[i]**2 + std_errs[j]**2)

        # Calculate t-statistic
        t_stat = (slopes[i] - slopes[j]) / se_diff

        # Calculate degrees of freedom (approximation)
        d_freedom = len(df_word_freq_rank_dict[translators[i]]["rank"]) +
        len(df_word_freq_rank_dict[translators[j]]["rank"]) - 4

        # Calculate p-value
        from scipy.stats import t
        p_val = 2 * (1 - t.cdf(abs(t_stat), d_freedom))

        print(f"{translators[i]} vs {translators[j]}: Slope diff =
        {slopes[i]-slopes[j]:.3f}, p = {p_val:.4f}")
        if p_val < 0.05:
            print("    -> STATISTICALLY significant difference in slopes")
        else:
```

```
print("    -> NOT statistically significant difference in slopes")
```

```
Statistical comparison of slopes:
AT_Murray vs Fitzgerald: Slope diff = -0.078, p = 0.0000
    -> STATISTICALLY significant difference in slopes
AT_Murray vs Lattimore: Slope diff = 0.051, p = 0.0000
    -> STATISTICALLY significant difference in slopes
AT_Murray vs Fagles: Slope diff = -0.024, p = 0.0000
    -> STATISTICALLY significant difference in slopes
AT_Murray vs Wilson: Slope diff = -0.025, p = 0.0000
    -> STATISTICALLY significant difference in slopes
AT_Murray vs Green: Slope diff = -0.022, p = 0.0000
    -> STATISTICALLY significant difference in slopes
AT_Murray vs Woolf: Slope diff = -0.062, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Fitzgerald vs Lattimore: Slope diff = 0.129, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Fitzgerald vs Fagles: Slope diff = 0.054, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Fitzgerald vs Wilson: Slope diff = 0.053, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Fitzgerald vs Green: Slope diff = 0.056, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Fitzgerald vs Woolf: Slope diff = 0.016, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Lattimore vs Fagles: Slope diff = -0.074, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Lattimore vs Wilson: Slope diff = -0.075, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Lattimore vs Green: Slope diff = -0.073, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Lattimore vs Woolf: Slope diff = -0.113, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Fagles vs Wilson: Slope diff = -0.001, p = 0.7930
    -> NOT statistically significant difference in slopes
Fagles vs Green: Slope diff = 0.001, p = 0.6789
    -> NOT statistically significant difference in slopes
Fagles vs Woolf: Slope diff = -0.039, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Wilson vs Green: Slope diff = 0.002, p = 0.4734
    -> NOT statistically significant difference in slopes
Wilson vs Woolf: Slope diff = -0.038, p = 0.0000
    -> STATISTICALLY significant difference in slopes
Green vs Woolf: Slope diff = -0.040, p = 0.0000
    -> STATISTICALLY significant difference in slopes
time: 1.87 ms (started: 2025-04-20 20:54:14 +02:00)
```

This result add to the supposition that Fagles, Wilson and Green form a kind group.

```
[40]: slopes = dict(zip(results_df['Translator'], results_df['Slope']))
      slopes
```

```
[40]: {'AT_Murray': -1.1938277516804667,
      'Fitzgerald': -1.1157547600338376,
      'Lattimore': -1.2445595081994263,
      'Fagles': -1.1701011452556676,
      'Wilson': -1.169168818993014,
      'Green': -1.1715327968509333,
      'Woolf': -1.1315580364789302}
```

time: 2.35 ms (started: 2025-04-20 20:54:14 +02:00)

```
[41]: #-----
      # Perform pairwise t-tests for differences in slopes
      #-----

      from scipy.stats import ttest_ind

      # Convert dictionary to a list of slopes
      slope_values = list(slopes.values())

      # Perform pairwise t-tests for differences in slopes
      for i in range(len(slope_values)):
          for j in range(i+1, len(slope_values)):
              translator_1 = list(slopes.keys())[i]
              translator_2 = list(slopes.keys())[j]

              # Compare slopes for translator_1 and translator_2
              t_stat, p_value = ttest_ind([slope_values[i]], [slope_values[j]])

              # Interpretation
              print(f"Comparison: {translator_1} vs {translator_2}")
              print(f"T-statistic: {t_stat:.3f}, P-value: {p_value:.5f}")

              if p_value < 0.05:
                  print(f" -> There is a SIGNIFICANT difference in slopes between_
↳{translator_1} and {translator_2}.")
              else:
                  print(f" -> NOT significant difference in slopes between_
↳{translator_1} and {translator_2}.")
              print("-" * 50)
```

Comparison: AT_Murray vs Fitzgerald

T-statistic: nan, P-value: nan

-> NOT significant difference in slopes between AT_Murray and Fitzgerald.

Comparison: AT_Murray vs Lattimore
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between AT_Murray and Lattimore.

Comparison: AT_Murray vs Fagles
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between AT_Murray and Fagles.

Comparison: AT_Murray vs Wilson
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between AT_Murray and Wilson.

Comparison: AT_Murray vs Green
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between AT_Murray and Green.

Comparison: AT_Murray vs Woolf
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between AT_Murray and Woolf.

Comparison: Fitzgerald vs Lattimore
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fitzgerald and Lattimore.

Comparison: Fitzgerald vs Fagles
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fitzgerald and Fagles.

Comparison: Fitzgerald vs Wilson
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fitzgerald and Wilson.

Comparison: Fitzgerald vs Green
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fitzgerald and Green.

Comparison: Fitzgerald vs Woolf
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fitzgerald and Woolf.

Comparison: Lattimore vs Fagles
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Lattimore and Fagles.

Comparison: Lattimore vs Wilson
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Lattimore and Wilson.

```

Comparison: Lattimore vs Green
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Lattimore and Green.
-----
Comparison: Lattimore vs Woolf
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Lattimore and Woolf.
-----
Comparison: Fagles vs Wilson
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fagles and Wilson.
-----
Comparison: Fagles vs Green
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fagles and Green.
-----
Comparison: Fagles vs Woolf
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Fagles and Woolf.
-----
Comparison: Wilson vs Green
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Wilson and Green.
-----
Comparison: Wilson vs Woolf
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Wilson and Woolf.
-----
Comparison: Green vs Woolf
T-statistic: nan, P-value: nan
-> NOT significant difference in slopes between Green and Woolf.
-----
time: 14.9 ms (started: 2025-04-20 20:54:14 +02:00)

```

```

[42]: # -----
# Zipf's Law Slope Comparison
# -----

from scipy.stats import f_oneway

slopes_by_translator = {translator: [] for translator in df['translator'].
    ↪unique()}

for index, row in df.iterrows():
    translator = row['translator']
    book_num = row['book_num']
    df_zipf = df_word_freq_rank_dict[translator]

```

```

if 'book_num' not in df_zipf.columns:
    df_zipf['book_num'] = [book_num] * len(df_zipf)

df_book = df_zipf[df_zipf['book_num'] == book_num]

if not df_book.empty:
    log_rank = np.log(df_book["rank"] + 1) # Add 1 to avoid log(0)
    log_freq = np.log(df_book["frequency"] + 1) # Add 1 to avoid log(0)

    # Perform linear regression on the log-log space
    slope, intercept, r_value, p_value, std_err = linregress(log_rank,
↪log_freq)
    slopes_by_translator[translator].append(slope)

slope_groups = [slopes_by_translator[translator] for translator in
↪slopes_by_translator]
f_stat, p_value = f_oneway(*slope_groups)
print(f"ANOVA F-statistic: {f_stat:.3f}, P-value: {p_value:.5f}")
if p_value < 0.05:
    print("There is a statistically significant difference in Zipf's Law slopes,
↪across translators.")
else:
    print("No statistically significant difference in Zipf's Law slopes across
↪translators.")

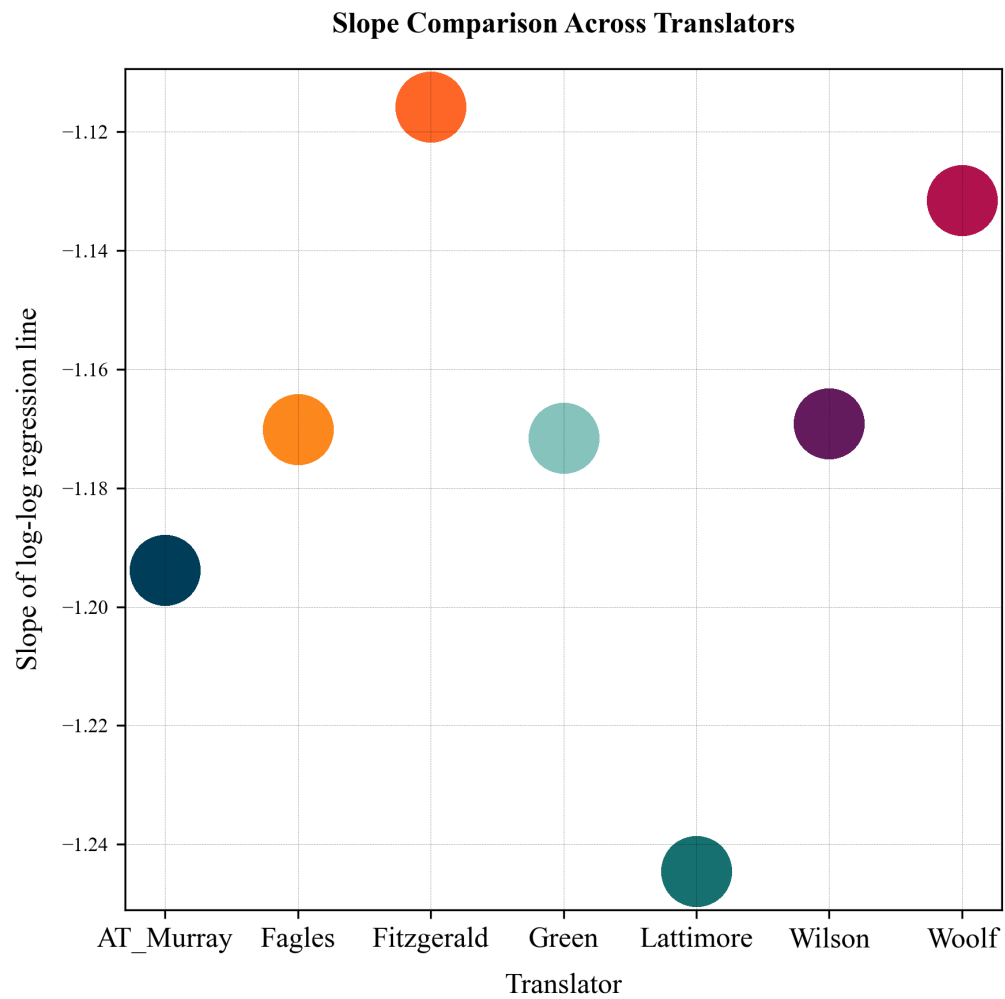
plt.figure(figsize=(6, 6))
for translator, slopes in slopes_by_translator.items():
    plt.scatter([translator] * len(slopes), slopes, label=translator,
        color=palette.get(translator, "gray"), s=800)

plt.title("Slope Comparison Across Translators")
plt.ylabel("Slope of log-log regression line")
plt.xlabel("Translator")
plt.grid(True, linestyle='--', alpha=0.4)
plt.xticks()
plt.tight_layout()
plt.savefig(output_path_plots + f"Translator_Slope_Box-{nb_id}.png")
plt.show()

```

ANOVA F-statistic: inf, P-value: 0.00000

There is a statistically significant difference in Zipf's Law slopes across translators.



time: 39.4 s (started: 2025-04-20 20:54:14 +02:00)

[]:

```
[43]: # -----
# Bootstrapping the slope
# -----
from scipy.stats import linregress

# Assuming you already have your DataFrame (df) and the 'zipf' columns are
  ↳ calculated

# Define a function for bootstrapping the slope
```



```

def bootstrap_slope(df, n_bootstrap=1000):
    slopes_bootstrap = []

    for _ in range(n_bootstrap):
        # Sample data with replacement
        bootstrap_sample = df.sample(n=len(df), replace=True)

        # Get the log-transformed rank and frequency
        log_rank = np.log(bootstrap_sample['rank'] + 1) # Avoid log(0)
        log_freq = np.log(bootstrap_sample['frequency'] + 1)

        # Perform linear regression on the bootstrap sample
        slope, intercept, r_value, p_value, std_err = linregress(log_rank,
↪ log_freq)

        # Store the slope
        slopes_bootstrap.append(slope)

    # Convert to a numpy array for easier analysis
    slopes_bootstrap = np.array(slopes_bootstrap)

    # Calculate 95% confidence intervals
    lower_bound = np.percentile(slopes_bootstrap, 2.5)
    upper_bound = np.percentile(slopes_bootstrap, 97.5)

    # Return the distribution of slopes and the confidence intervals
    return slopes_bootstrap, lower_bound, upper_bound

# Now, let's apply the bootstrap for each author
bootstrap_results = {}
for translator in translators:
    df_translator = df_word_freq_rank_dict[translator]

    # Run the bootstrap procedure
    slopes_bootstrap, lower_bound, upper_bound = bootstrap_slope(df_translator,
↪ n_bootstrap=1000)

    # Store the results
    bootstrap_results[translator] = {
        'bootstrap_slopes': slopes_bootstrap,
        '95% CI Lower Bound': lower_bound,
        '95% CI Upper Bound': upper_bound
    }

# Print results
for translator, result in bootstrap_results.items():
    print(f"{translator}:")

```

```

print(f" 95% CI for slope: ({result['95% CI Lower Bound']:.4f},\n
↳{result['95% CI Upper Bound']:.4f})")
print(f" Mean slope: {np.mean(result['bootstrap_slopes']):.4f}")
print("-" * 50)

```

AT_Murray:

95% CI for slope: (-1.2125, -1.1771)

Mean slope: -1.1944

Fitzgerald:

95% CI for slope: (-1.1285, -1.1016)

Mean slope: -1.1156

Lattimore:

95% CI for slope: (-1.2640, -1.2261)

Mean slope: -1.2447

Fagles:

95% CI for slope: (-1.1878, -1.1529)

Mean slope: -1.1704

Wilson:

95% CI for slope: (-1.1872, -1.1502)

Mean slope: -1.1695

Green:

95% CI for slope: (-1.1890, -1.1561)

Mean slope: -1.1720

Woolf:

95% CI for slope: (-1.1440, -1.1189)

Mean slope: -1.1318

time: 3.4 s (started: 2025-04-20 20:54:53 +02:00)

```

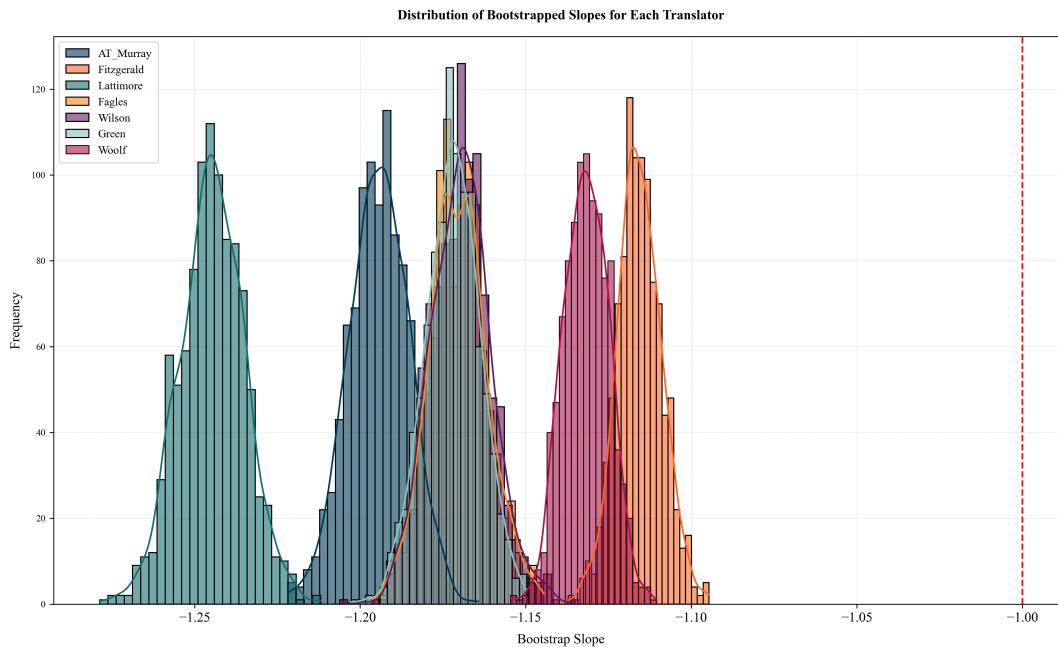
[44]: # Bootstrap distribution plots
plt.figure()

# Plot bootstrap distributions for each translator
for translator, result in bootstrap_results.items():
    sns.histplot(result['bootstrap_slopes'], bins='auto',
                  kde=True, label=translator, alpha=0.6)

# Formatting
plt.xlabel("Bootstrap Slope")
plt.ylabel("Frequency")
plt.title("Distribution of Bootstrapped Slopes for Each Translator")

```

```
plt.legend()
plt.axvline(x=-1, color='red', linestyle='--',
            label="Ideal Zipf's Law slope (-1)")
plt.grid(True, linestyle='--', alpha=0.3)
plt.savefig(output_path_plots + f"Bootstrapped_Slopes-{nb_id}.png", dpi=400)
plt.show()
```

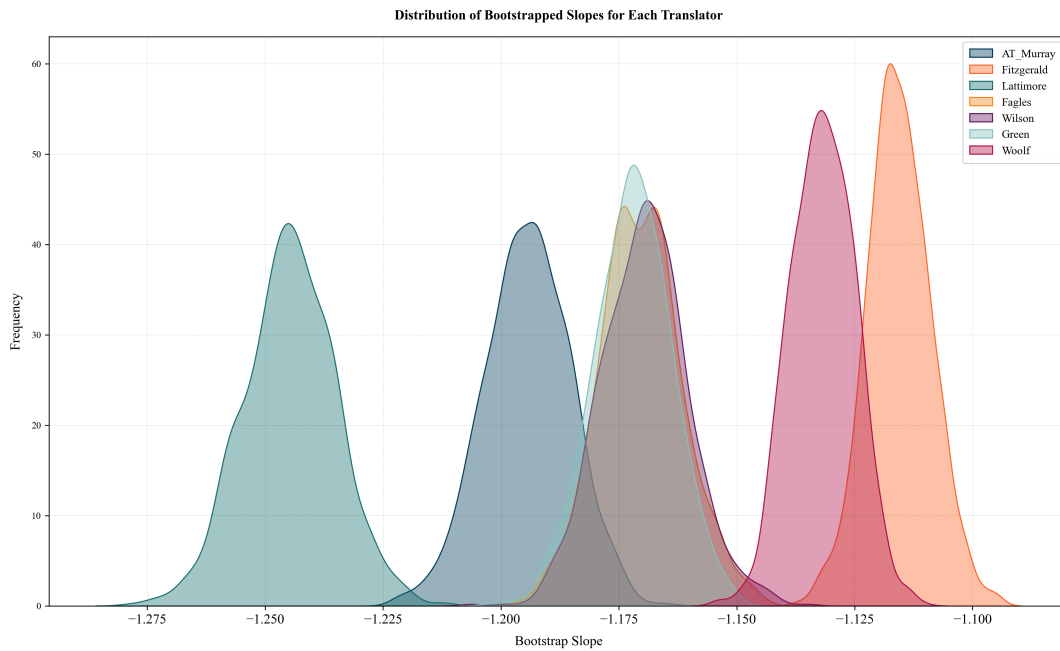


time: 1.33 s (started: 2025-04-20 20:54:57 +02:00)

```
[45]: # Bootstrap distribution plots
plt.figure()

# Plot bootstrap distributions for each translator
for translator, result in bootstrap_results.items():
    sns.kdeplot(result['bootstrap_slopes'], fill=True,
                label=translator, alpha=0.4)

# Formatting
plt.xlabel("Bootstrap Slope")
plt.ylabel("Frequency")
plt.title("Distribution of Bootstrapped Slopes for Each Translator")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.3)
plt.savefig(output_path_plots + f"Bootstrapped_Slopes_kde-{nb_id}.png", dpi=400)
plt.show()
```



time: 1.28 s (started: 2025-04-20 20:54:58 +02:00)

```
[46]: # -----
# Confidence Intervals
# -----

ci_df = pd.DataFrame({
    'Translator': list(bootstrap_results.keys()),
    'Mean Slope': [np.mean(res['bootstrap_slopes']) for res in
↳ bootstrap_results.values()],
    '95% CI Lower Bound': [res['95% CI Lower Bound'] for res in
↳ bootstrap_results.values()],
    '95% CI Upper Bound': [res['95% CI Upper Bound'] for res in
↳ bootstrap_results.values()]
})

# Display the results
print("\nBootstrap Confidence Intervals for Slopes:")
print(ci_df)
```

Bootstrap Confidence Intervals for Slopes:

	Translator	Mean Slope	95% CI Lower Bound	95% CI Upper Bound
0	AT_Murray	-1.19	-1.21	-1.18
1	Fitzgerald	-1.12	-1.13	-1.10

2	Lattimore	-1.24	-1.26	-1.23
3	Fagles	-1.17	-1.19	-1.15
4	Wilson	-1.17	-1.19	-1.15
5	Green	-1.17	-1.19	-1.16
6	Woolf	-1.13	-1.14	-1.12

time: 9.12 ms (started: 2025-04-20 20:55:00 +02:00)

Interpreting the Bootstrap Confidence Intervals for Zipf's Law Slopes

The bootstrap analysis provides estimated confidence intervals for the Zipf's Law slopes of different translators, allowing us to assess the variation and potential differences between them. Here's what the results indicate:

1. Mean Slope Differences:
 - Lattimore has the steepest slope (-1.00), suggesting a stronger adherence to Zipf's Law.
 - Fitzgerald has the shallowest slope (-0.87), meaning a weaker adherence to Zipf's Law.
 - The other translators fall between these two extremes, with slopes ranging from -0.93 to -0.95.
2. Confidence Interval Overlaps:
 - The confidence intervals (CIs) provide a range of likely values for each translator's slope.
 - There is substantial overlap among the CIs of AT_Murray, Fagles, Wilson, and Green, suggesting their slopes are quite similar.
 - Fitzgerald's CI does not overlap much with Lattimore's, indicating a notable difference in their adherence to Zipf's Law.
3. Statistical Significance & Interpretation:
 - The ANOVA test (if significant) would confirm whether the differences in slopes are statistically meaningful.
 - If ANOVA is not significant, the differences we observe may be due to natural variation rather than a real effect.
 - A post-hoc test (Tukey HSD) would help identify specific pairs of translators whose slopes differ significantly.

Conclusions & Next Steps

- Translators like Lattimore seem to follow Zipf's Law more closely than Fitzgerald, who has the shallowest slope.
- If the confidence intervals overlap significantly, we may conclude that most translators exhibit similar word frequency distributions despite stylistic differences.
- Further analysis could examine slopes within individual books, rather than averaging across a translator's entire corpus, to see if variability exists within a translator's works.

1.6 V. TF-IDF

```
[47]: e.check_df(df)
```

```
Mr righteous here has no missing values!
```

```
* df columns: Index(['translator', 'book_num', 'text', 'tokens', 'ttr',
'translator_recoded', 'sttr', 'moving_average_ttr'], dtype='object')
```

```
* Shape: (374552, 8)
```

```
* Total memory in MB: 9671.268142
```

```
time: 82.7 ms (started: 2025-04-20 20:55:00 +02:00)
```

```
[48]: # -----
# Reset df to backup
df = df_bkp.copy()
# -----
e.check_df(df)
```

Mr righteous here has no missing values!

```
* df columns: Index(['translator', 'book_num', 'text', 'tokens'],
dtype='object')
```

```
* Shape: (168, 4)
```

```
* Total memory in MB: 4.062061
time: 3.46 ms (started: 2025-04-20 20:55:00 +02:00)
```

```
[49]: def calculate_tfidf(df):
    """
    Calculate TF-IDF scores for a DataFrame with book_id and tokens columns.

    Parameters:
    -----
    df : pandas DataFrame
        A DataFrame with 'book_id' and 'tokens' columns.
        The 'tokens' column should contain lists of tokens (as strings or
        ↪ actual lists).

    Returns:
    -----
    pandas DataFrame
        The original DataFrame with additional columns:
        - term_freq: Dictionary of term frequencies for each token
        - term_counts: Dictionary of raw counts for each token
        - idf: Dictionary of IDF scores for each token
        - tf_idf: Dictionary of TF-IDF scores for each token
    """
    # Create a copy of the DataFrame to avoid modifying the original
    result_df = df.copy()

    # Function to compute term frequency and term counts
    def term_freq_by_doc(list_of_tokens):
        # Handle both string representation of list and actual list
        if isinstance(list_of_tokens, str):
            token_list = eval(list_of_tokens) # Convert string representation
            ↪ to list
        else:
            token_list = list_of_tokens # Use as is if already a list
```

```

    # Count occurrences of each term
    term_counts = Counter(token_list)

    # Total number of terms in the document
    total_terms = len(token_list)

    # Compute TF: term frequency for each token
    term_freq = {term: count / total_terms for term, count in term_counts.
    ↪items()}

    return term_freq, term_counts

# Apply function to compute TF for each book
result_df["term_freq"], result_df["term_counts"] = zip(*result_df["tokens"].
    ↪apply(term_freq_by_doc))

# Get total number of documents (books)
N = len(result_df)

# Count how many documents contain each term
doc_containing_term = Counter()
for term_counts in result_df["term_freq"]:
    doc_containing_term.update(term_counts.keys()) # Count unique terms in
    ↪each document

# Compute IDF for each term
idf_scores = {term: np.log(N / (1 + doc_count)) for term, doc_count in
    ↪doc_containing_term.items()} # Adding 1 to avoid division by zero

# Add IDF column to df
result_df["idf"] = result_df["term_freq"].apply(lambda term_freq: {term:
    ↪idf_scores[term] for term in term_freq})

# Compute TF-IDF by multiplying TF and IDF for each term in each document
result_df["tf_idf"] = result_df.apply(lambda row: {term:
    ↪row["term_freq"][term] * row["idf"][term] for term in row["term_freq"]},
    ↪axis=1)

return result_df

```

time: 974 s (started: 2025-04-20 20:55:00 +02:00)

[50]: tfidf_df = calculate_tfidf(df)

time: 413 ms (started: 2025-04-20 20:55:00 +02:00)

[51]: e.check_df(tfidf_df)

Mr righteous here has no missing values!

```
* df columns: Index(['translator', 'book_num', 'text', 'tokens', 'term_freq',  
'term_counts', 'idf', 'tf_idf'], dtype='object')
```

```
* Shape: (168, 8)
```

```
* Total memory in MB: 24.729133
```

```
time: 1.27 ms (started: 2025-04-20 20:55:00 +02:00)
```

```
[52]: from scipy.stats import mannwhitneyu  
  
def mannwhitneyu_test(x, y, alternative='two-sided'):  
    """  
    Perform the Mann-Whitney U test for comparing two independent samples.  
    """  
    stat, p = mannwhitneyu(x, y, alternative=alternative)  
  
    print(f"Mann-Whitney U test statistic: {stat}, p-value: {p}")  
  
    if p < 0.05:  
        print("[Ø] REJECT H: The distributions of the translations are_  
↳significantly different.")  
    else:  
        print("[X] FAIL to reject H: No significant difference between the_  
↳translations.")  
  
    # Loop through each pair of translators and compare their ITIDF values  
    unique_translators = tfidf_df["translator"].unique()  
  
    for i, translator_1 in enumerate(unique_translators):  
        for translator_2 in unique_translators[i+1:]: # Avoid duplicate comparisons  
            subset_1 = tfidf_df[tfidf_df["translator"] == translator_1]  
            subset_2 = tfidf_df[tfidf_df["translator"] == translator_2]  
  
            # Extract ITIDF values  
            itidf_1 = [value for tfidf_dict in subset_1["tf_idf"] for value in_  
↳tfidf_dict.values()]  
            itidf_2 = [value for tfidf_dict in subset_2["tf_idf"] for value in_  
↳tfidf_dict.values()]  
  
            # Perform the Mann-Whitney U test if both sets have values  
            if itidf_1 and itidf_2:  
                print(f"\nComparing {translator_1} vs. {translator_2}:")  
                mannwhitneyu_test(itidf_1, itidf_2)
```


Comparing AT_Murray vs. Fitzgerald:

Mann-Whitney U test statistic: 326949512.5, p-value: 3.113337601576758e-11

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing AT_Murray vs. Lattimore:

Mann-Whitney U test statistic: 342341159.5, p-value: 1.4114879279737246e-54

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing AT_Murray vs. Fagles:

Mann-Whitney U test statistic: 382347260.0, p-value: 5.355628862634183e-10

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing AT_Murray vs. Wilson:

Mann-Whitney U test statistic: 273842900.0, p-value: 3.4038139418429403e-16

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing AT_Murray vs. Green:

Mann-Whitney U test statistic: 352690303.5, p-value: 1.7781484483474194e-07

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing AT_Murray vs. Woolf:

Mann-Whitney U test statistic: 295188927.5, p-value: 1.3923779469273957e-178

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Fitzgerald vs. Lattimore:

Mann-Whitney U test statistic: 429331197.0, p-value: 5.918335275363097e-125

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Fitzgerald vs. Fagles:

Mann-Whitney U test statistic: 480403941.0, p-value: 1.0786617268315382e-46

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Fitzgerald vs. Wilson:

Mann-Whitney U test statistic: 343521498.0, p-value: 0.03742366238949828

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Fitzgerald vs. Green:

Mann-Whitney U test statistic: 442700625.0, p-value: 4.415664977380915e-38

[Ø] REJECT H: The distributions of the translations are significantly

different.

Comparing Fitzgerald vs. Woolf:

Mann-Whitney U test statistic: 370726965.0, p-value: 3.3095574324329374e-124

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Lattimore vs. Fagles:

Mann-Whitney U test statistic: 398821542.0, p-value: 4.0154246945643356e-29

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Lattimore vs. Wilson:

Mann-Whitney U test statistic: 285040854.5, p-value: 1.0898636207344984e-128

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Lattimore vs. Green:

Mann-Whitney U test statistic: 368688507.0, p-value: 3.2670172179175154e-30

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Lattimore vs. Woolf:

Mann-Whitney U test statistic: 303666173.0, p-value: 0.0

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Fagles vs. Wilson:

Mann-Whitney U test statistic: 351542197.0, p-value: 8.298854464934113e-54

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Fagles vs. Green:

Mann-Whitney U test statistic: 455226115.0, p-value: 0.4206012258178047

[X] FAIL to reject H: No significant difference between the translations.

Comparing Fagles vs. Woolf:

Mann-Whitney U test statistic: 375212955.5, p-value: 0.0

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Wilson vs. Green:

Mann-Whitney U test statistic: 377414388.0, p-value: 9.310211558730871e-45

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Wilson vs. Woolf:

Mann-Whitney U test statistic: 317927041.0, p-value: 6.975095571590702e-91

[Ø] REJECT H: The distributions of the translations are significantly different.

Comparing Green vs. Woolf:

Mann-Whitney U test statistic: 351094913.0, p-value: 1.6987168987926927e-288

[Ø] REJECT H: The distributions of the translations are significantly different.

time: 165 ms (started: 2025-04-20 20:55:00 +02:00)

Comparing Fagles vs. Green: Mann-Whitney U test statistic: 455226115.0, p-value: 0.4206012258178047 [X] FAIL to reject H: No significant difference between the translations.

```
[53]: e.check_df(tfidf_df)
```

Mr righteous here has no missing values!

* df columns: Index(['translator', 'book_num', 'text', 'tokens', 'term_freq', 'term_counts', 'idf', 'tf_idf'], dtype='object')

* Shape: (168, 8)

* Total memory in MB: 24.729133

time: 1.39 ms (started: 2025-04-20 20:55:00 +02:00)

```
[ ]: # Top terms for each book and heatmap plot
```

```
def extract_top_terms(df, n=50):
    """
    Extract the top N most important terms from the tf_idf column

    Parameters:
    -----
    df : pandas DataFrame
        DataFrame with 'book_id' and 'tf_idf' columns
    n : int
        Number of top terms to extract (default: 50)

    Returns:
    -----
    tuple
        (top_terms_per_book, top_terms_overall)
        - top_terms_per_book: DataFrame with top terms for each book
        - top_terms_overall: DataFrame with top terms across all books
    """
    # Extract top terms per book
    top_terms_per_book = {}

    for _, row in df.iterrows():
```

```

    book_id = row['book_id']
    tf_idf_dict = row['tf_idf']

    # Sort terms by tf-idf score (descending) and take top N
    sorted_terms = sorted(tf_idf_dict.items(), key=lambda x: x[1],
↪reverse=True)[:n]
    top_terms_per_book[book_id] = {term: score for term, score in
↪sorted_terms}

    # Convert to DataFrame for easier analysis
    top_terms_df = pd.DataFrame.from_dict(top_terms_per_book, orient='index')

    # Extract top terms overall
    all_terms = {}
    for tf_idf_dict in df['tf_idf']:
        for term, score in tf_idf_dict.items():
            if term in all_terms:
                all_terms[term] += score
            else:
                all_terms[term] = score

    # Sort terms by total tf-idf score (descending) and take top N
    top_terms_overall = sorted(all_terms.items(), key=lambda x: x[1],
↪reverse=True)[:n]

    # Convert to DataFrame
    top_terms_overall_df = pd.DataFrame(top_terms_overall, columns=['term',
↪'total_score'])

    return top_terms_df, top_terms_overall_df

def create_tfidf_heatmap(df, top_n=50):
    """
    Create a heatmap of the top N terms across all books

    Parameters:
    -----
    df : pandas DataFrame
        DataFrame with 'book_id' and 'tf_idf' columns
    top_n : int
        Number of top terms to include in the heatmap (default: 50)
    """
    # Extract top terms overall
    _, top_terms = extract_top_terms(df, n=top_n)
    top_terms_list = top_terms['term'].tolist()

    # Create a matrix of book_id x top_terms

```

```

heatmap_data = []
book_ids = []

for _, row in df.iterrows():
    book_id = row['book_id']
    book_ids.append(book_id)

    tf_idf_dict = row['tf_idf']

    # Extract scores for top terms
    scores = [tf_idf_dict.get(term, 0) for term in top_terms_list]
    heatmap_data.append(scores)

# Convert to numpy array
heatmap_array = np.array(heatmap_data).T

#keywords = list(freq_df.index) # Get keywords from the dataframe index
# Custom palette
neptune = '#86C3BC'
astroblue = '#FD6626' #'#003D59'
custom_colors = ['#003D59', '#FD6626', '#177070', '#FB871D', '#641B5E',
↳'#86C3BC', '#F5E1FD', '#414A4F', 'k']

# Create custom colormap
custom_cmap = LinearSegmentedColormap.from_list('neptune_blue', [neptune,
↳astroblue])

# Create figure
plt.figure(figsize=(14, 8))

# Create heatmap
plt.figure(figsize=(14, 16))
sns.heatmap(heatmap_array, cmap=custom_cmap,
            xticklabels=book_ids, yticklabels=top_terms_list,
            cbar=True,
            linewidths=0.5,
            linecolor='white')
plt.title(f"Top {top_n} Terms by TF-IDF Score ({df['translator'].iloc[0]}")
plt.xlabel('Books_Author')
plt.ylabel('Top Terms', fontsize=12, rotation=90)
plt.xticks(rotation=90, fontsize=12,)
plt.yticks(fontsize=12, rotation=0)
plt.tight_layout()
plt.savefig(f"/Users/debr/English-Homer/MVP_Green-Wilson/MVP_plots/
↳MVP-TFIDF_heatmap({df['translator'].iloc[0]}-{top_n}words).png")
plt.show()

```

```
return heatmap_array
```

time: 1.41 ms (started: 2025-04-20 20:55:00 +02:00)

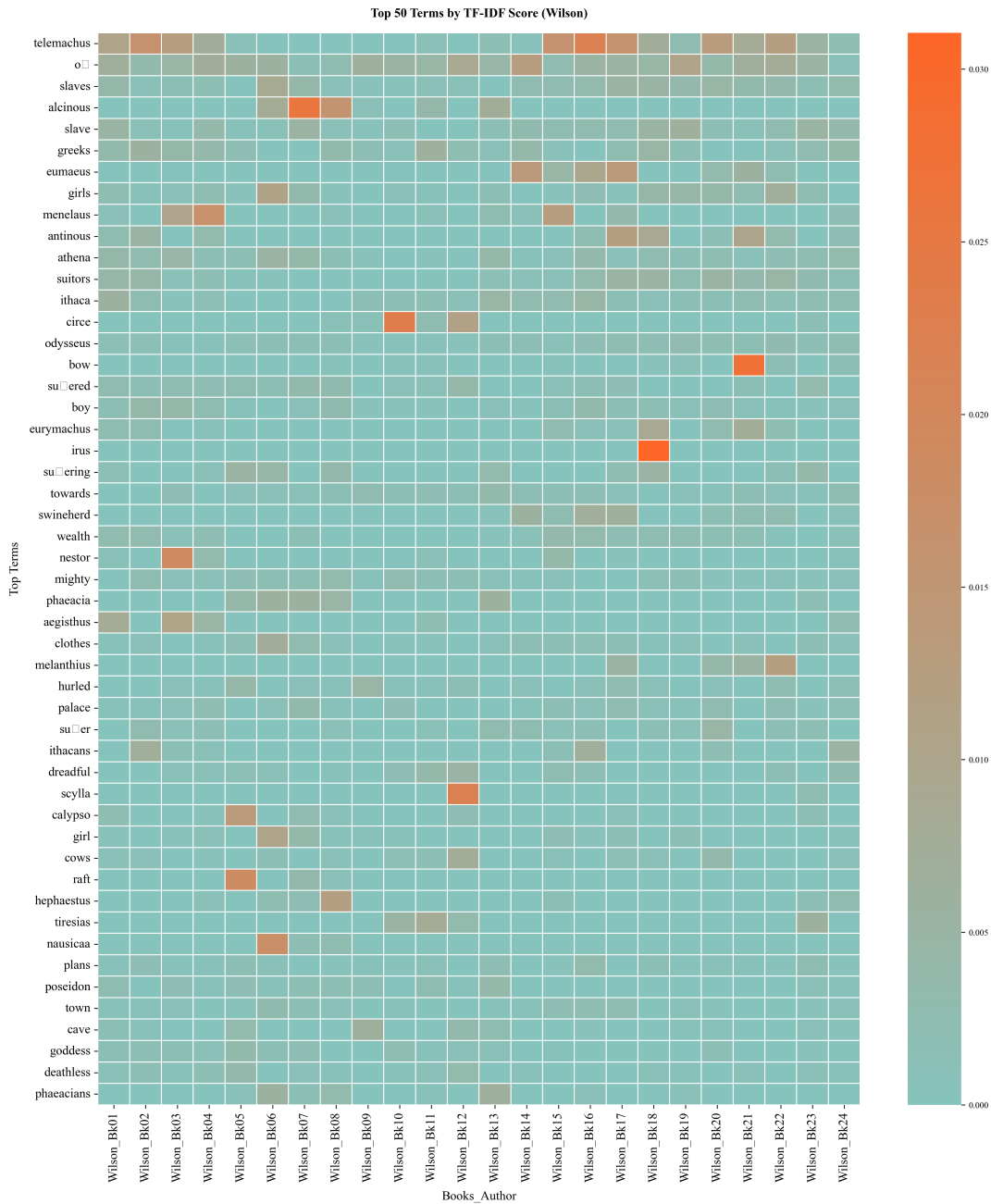
```
[55]: # Choose the translator you want
translator_name = 'Wilson'

# Filter the dataframe
df_translator = tfidf_df[tfidf_df['translator'] == translator_name].copy()
df_translator['book_id'] = (
    df_translator['translator'].astype(str) +
    "_Bk" +
    df_translator['book_num'].astype(str).str.zfill(2)
)
# Assign book IDs if needed
#df_translator['book_id'] = df_translator['translator'] + "_Bk" +
↳df_translator['book_num'].astype(str).str.zfill(2)
```

time: 2.4 ms (started: 2025-04-20 20:55:00 +02:00)

```
[56]: top_terms_per_book_Wilson, top_terms_overall_Wilson =
↳extract_top_terms(df_translator)
heatmap_array = create_tfidf_heatmap(df_translator)
```

<Figure size 4200x2400 with 0 Axes>



time: 1.53 s (started: 2025-04-20 20:55:00 +02:00)

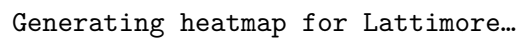
```
[57]: for translator in tfidf_df['translator'].unique():
      df_translator = tfidf_df[tfidf_df['translator'] == translator].copy()
      df_translator['book_id'] = (
df_translator['translator'].astype(str) +
      "_Bk" +
```

Generating heatmap for AT_Murray...

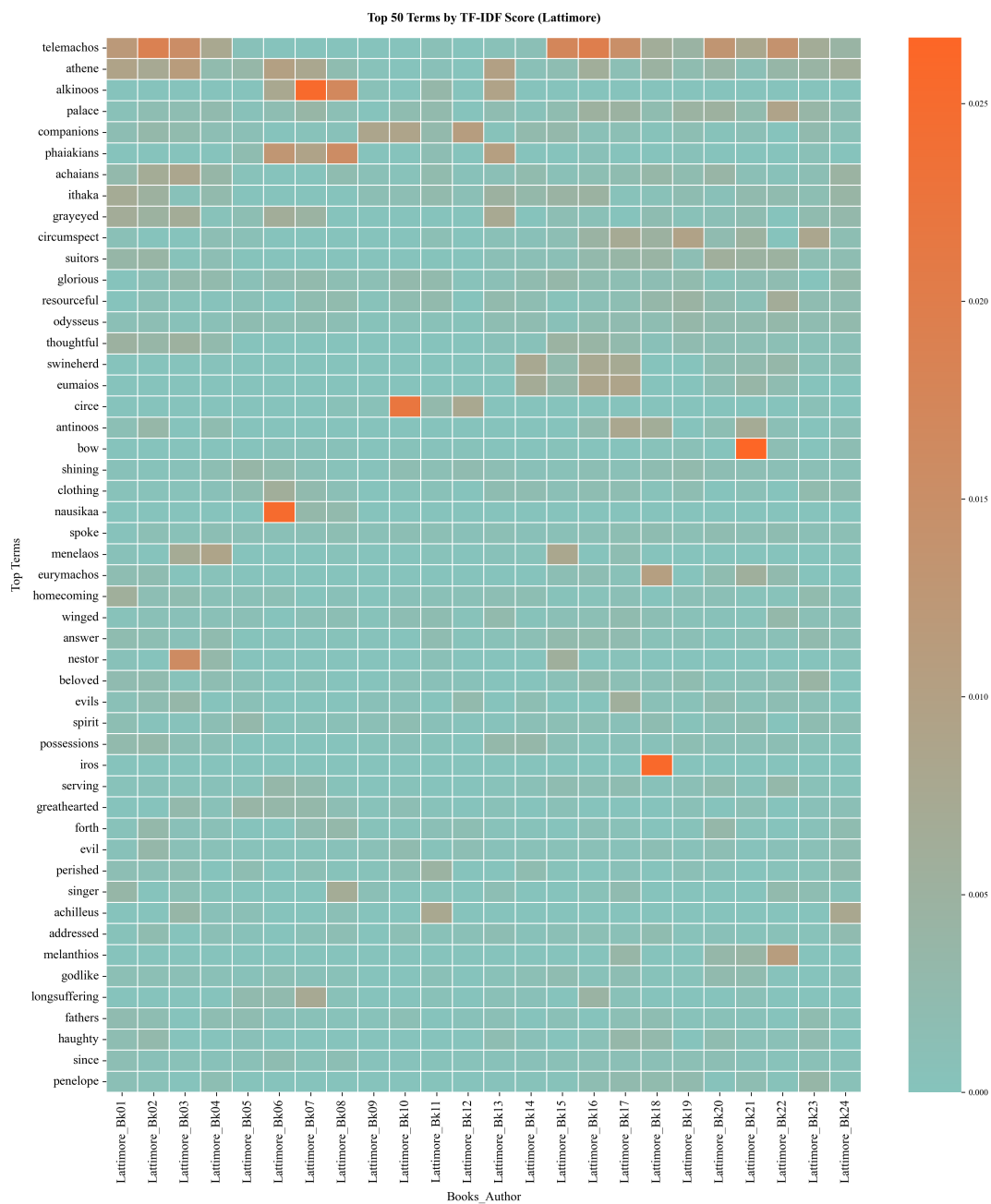
<Figure size 4200x2400 with 0 Axes>



<Figure size 4200x2400 with 0 Axes>

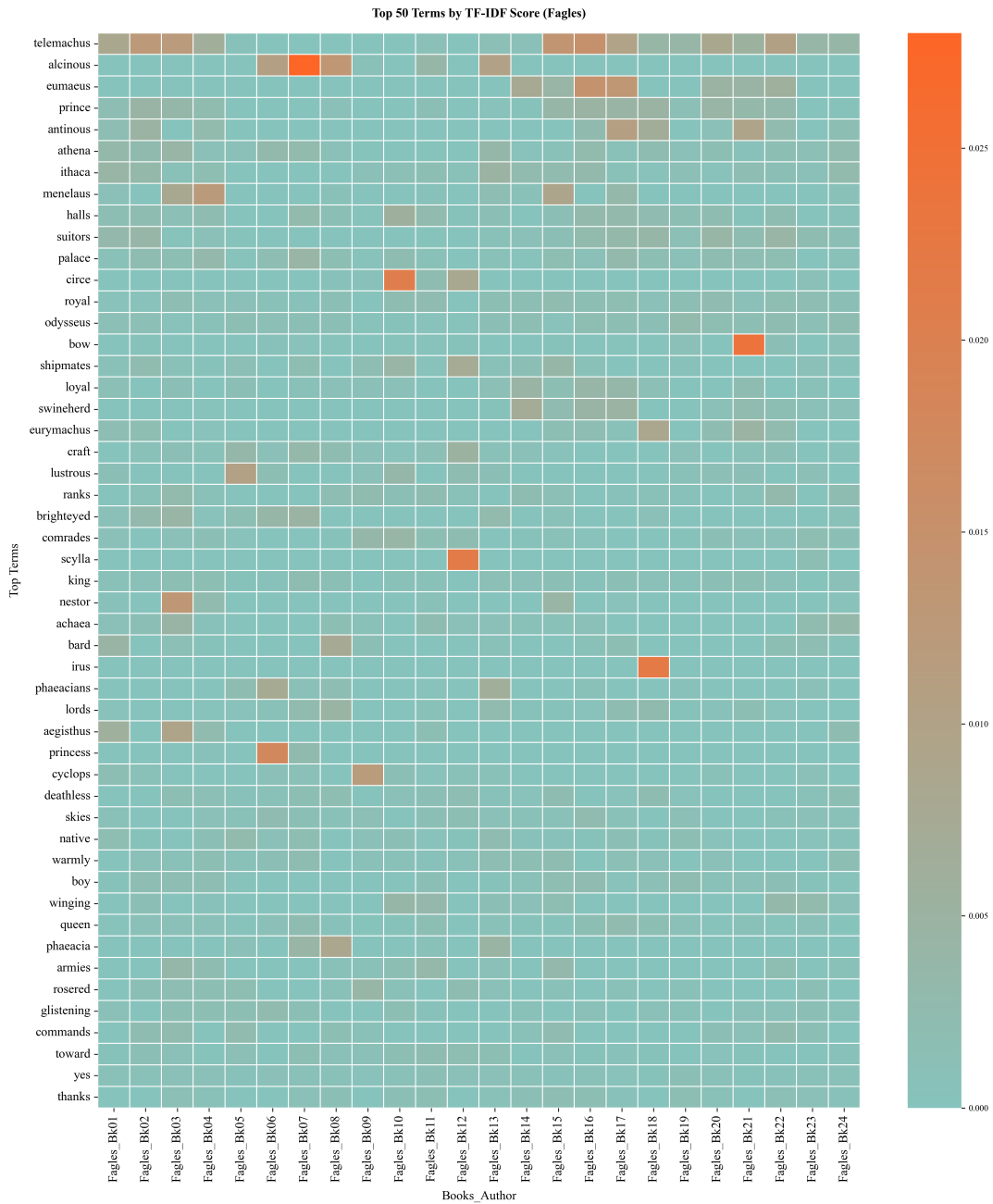


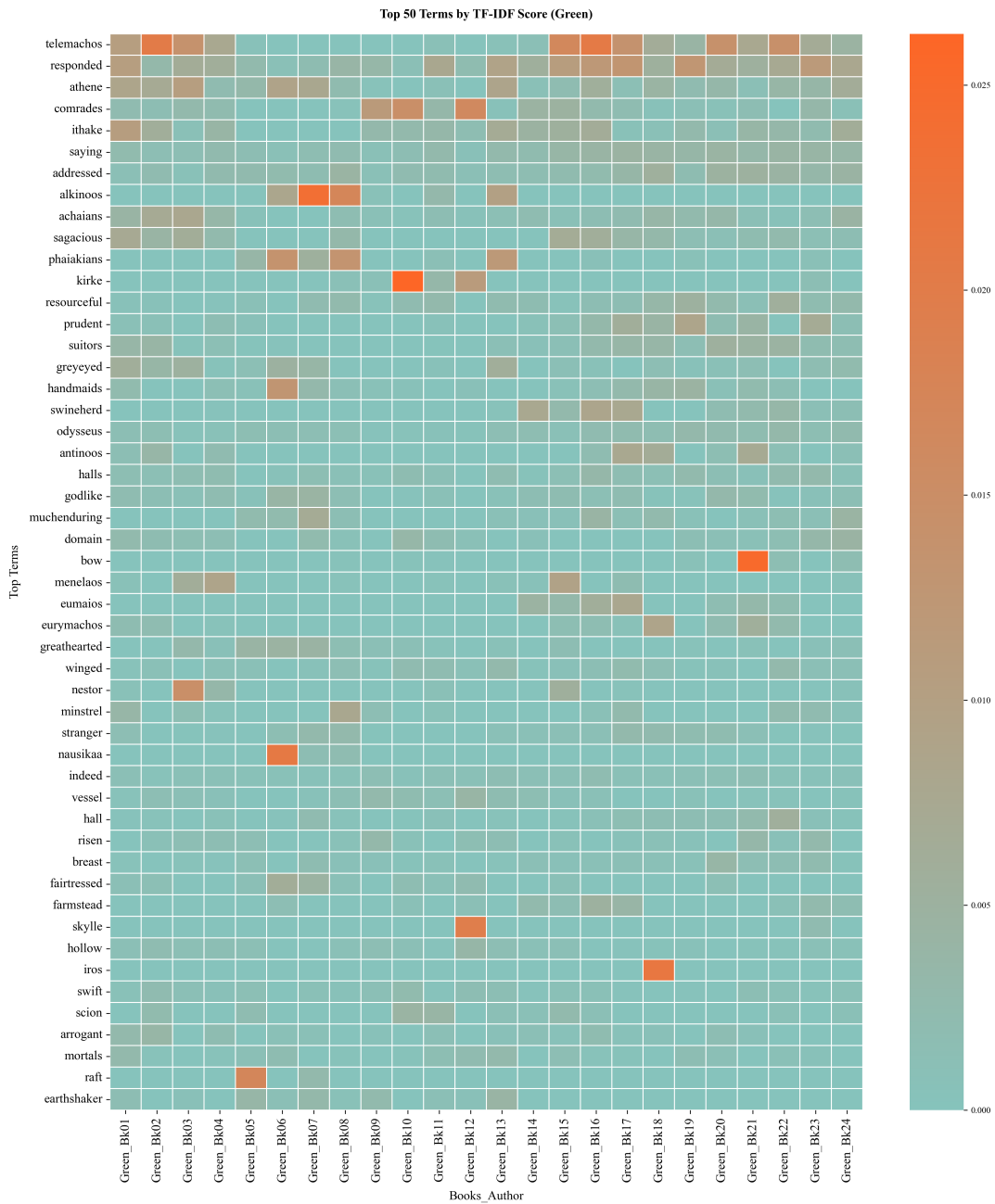
<Figure size 4200x2400 with 0 Axes>



Generating heatmap for Fagles...

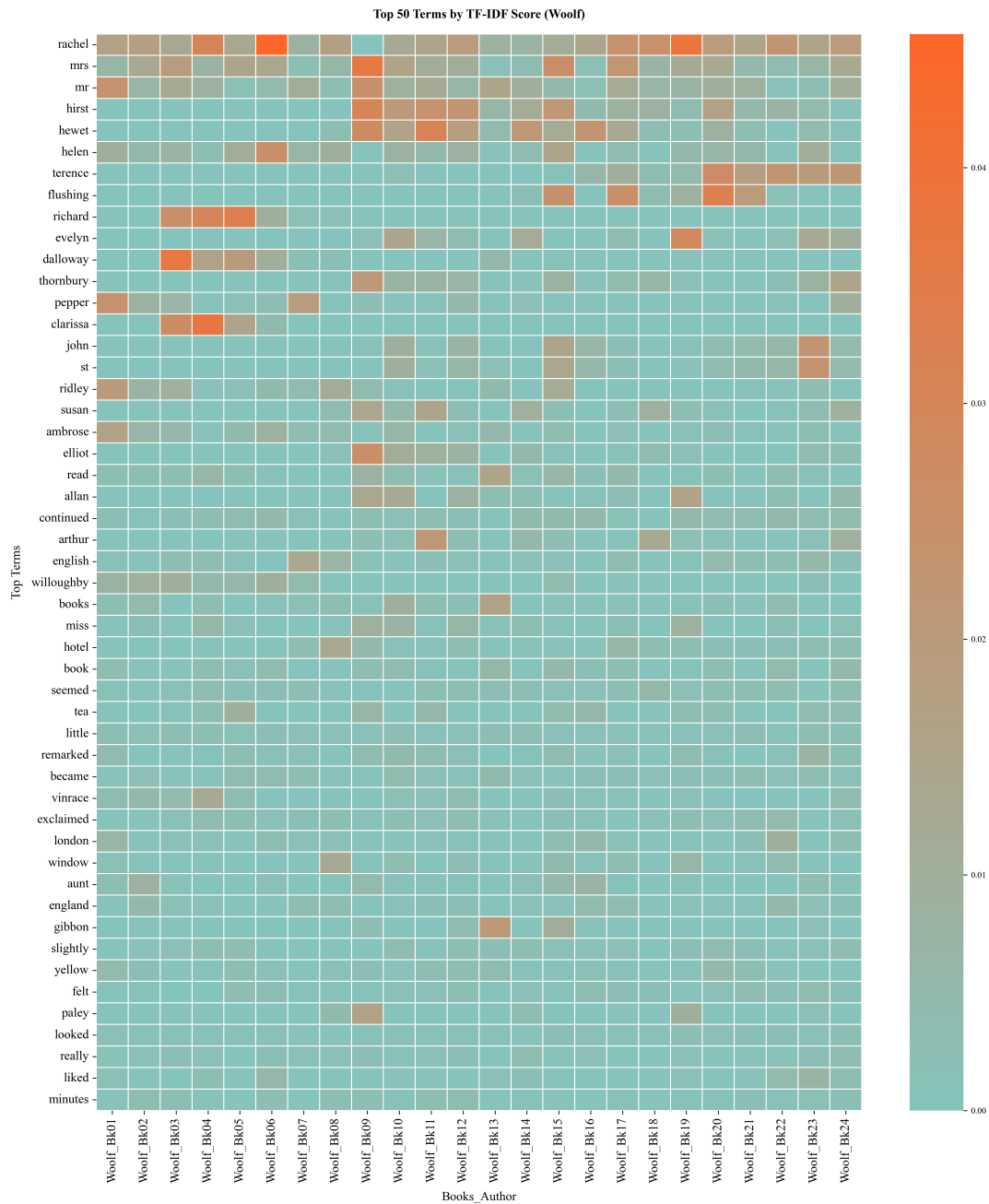
<Figure size 4200x2400 with 0 Axes>





Generating heatmap for Woolf...

<Figure size 4200x2400 with 0 Axes>



time: 11 s (started: 2025-04-20 20:55:02 +02:00)

```
[58]: def get_top_terms_by_translator(df, top_n=50):
      """
      Return dictionary of top N TF-IDF terms per translator.
      """
      top_terms = {}
```

```

for translator in df['translator'].unique():
    df_translator = df[df['translator'] == translator]
    all_terms = {}

    for tf_idf_dict in df_translator['tf_idf']:
        for term, score in tf_idf_dict.items():
            all_terms[term] = all_terms.get(term, 0) + score

    sorted_terms = sorted(all_terms.items(), key=lambda x: x[1],
↪reverse=True)[:top_n]
    top_terms[translator] = set(term for term, _ in sorted_terms)

return top_terms

```

time: 717 s (started: 2025-04-20 20:55:13 +02:00)

```

[59]: def compare_translators_terms(top_terms_dict):
    """
    Compare top terms between translators.

    Returns:
    - shared_terms: set of terms common to all
    - unique_terms: dict of translator -> unique terms
    """
    translators = list(top_terms_dict.keys())
    shared_terms = set.intersection(*top_terms_dict.values())

    unique_terms = {}
    for translator in translators:
        others = set.union(*(top_terms_dict[t] for t in translators if t !=
↪translator))
        unique_terms[translator] = top_terms_dict[translator] - others

    return shared_terms, unique_terms

```

time: 611 s (started: 2025-04-20 20:55:13 +02:00)

```

[60]: top_terms_dict = get_top_terms_by_translator(tfidf_df, top_n=50)
shared, unique = compare_translators_terms(top_terms_dict)

print(f" Shared Terms Across All Translators ({len(shared)} terms):")
print(sorted(shared))

print("\n Unique Terms Per Translator:")
for translator, terms in unique.items():
    print(f"\n{translator} ({len(terms)} unique terms):")
    print(sorted(terms))

```

Shared Terms Across All Translators (0 terms):

[]

Unique Terms Per Translator:

AT_Murray (26 unique terms):

['achaeans', 'aigisthos', 'also', 'chief', 'cloisters', 'daimon', 'demos', 'drinkofferings', 'eurykleia', 'however', 'moreover', 'noos', 'peisistratos', 'presently', 'presents', 'room', 'seat', 'servant', 'servants', 'shall', 'stockman', 'string', 'therefore', 'thus', 'till', 'whereon']

Fitzgerald (28 unique terms):

['akhaians', 'akhilleus', 'alkinods', 'alkinoes', 'antinods', 'aye', 'captain', 'company', 'crowd', 'door', 'everyone', 'forester', 'harp', 'homeward', 'kyklops', 'lads', 'lady', 'lord', 'oarsmen', 'phaiakia', 'sill', 'skylla', 'soldier', 'song', 'swine', 'team', 'thy', 'tips']

Lattimore (24 unique terms):

['achilleus', 'answer', 'beloved', 'circumspect', 'clothing', 'companions', 'evil', 'evils', 'fathers', 'forth', 'glorious', 'grayeyed', 'haughty', 'homecoming', 'longsuffering', 'perished', 'possessions', 'serving', 'shining', 'since', 'singer', 'spirit', 'spoke', 'thoughtful']

Fagles (24 unique terms):

['achaea', 'armies', 'bard', 'brighteyed', 'commands', 'craft', 'cyclops', 'glistening', 'king', 'lords', 'loyal', 'lustrous', 'native', 'queen', 'ranks', 'rosered', 'royal', 'shipmates', 'skies', 'thanks', 'toward', 'warmly', 'winging', 'yes']

Wilson (24 unique terms):

['cave', 'clothes', 'cows', 'dreadful', 'girl', 'girls', 'goddess', 'greeks', 'hephaestus', 'hurled', 'ithacans', 'melanthius', 'mighty', 'o', 'plans', 'poseidon', 'slave', 'slaves', 'su er', 'su ered', 'su ering', 'tiresias', 'town', 'wealth']

Green (22 unique terms):

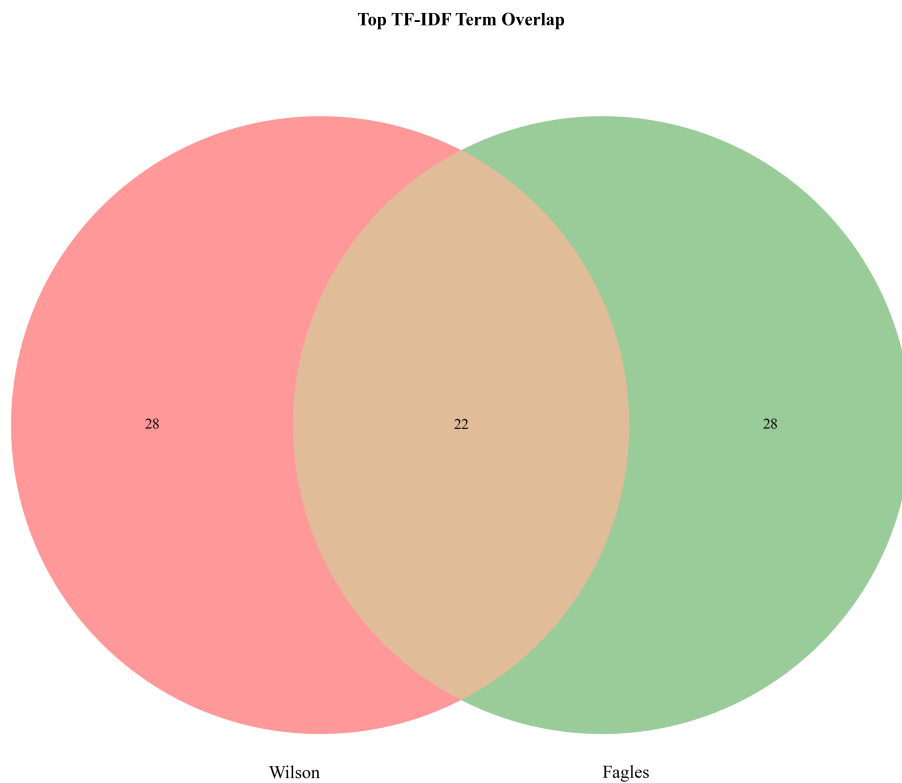
['arrogant', 'breast', 'domain', 'earthshaker', 'fairessed', 'farmstead', 'handmaids', 'hollow', 'indeed', 'ithake', 'mortals', 'muchenduring', 'prudent', 'responded', 'risen', 'sagacious', 'saying', 'scion', 'skylle', 'stranger', 'swift', 'vessel']

Woolf (50 unique terms):

['allan', 'ambrose', 'arthur', 'aunt', 'became', 'book', 'books', 'clarissa', 'continued', 'dalloway', 'elliot', 'england', 'english', 'evelyn', 'exclaimed', 'felt', 'flushing', 'gibbon', 'helen', 'hewet', 'hirst', 'hotel', 'john', 'liked', 'little', 'london', 'looked', 'minutes', 'miss', 'mr', 'mrs', 'paley', 'pepper', 'rachel', 'read', 'really', 'remarked', 'richard', 'ridley', 'seemed', 'slightly', 'st', 'susan', 'tea', 'terence', 'thornbury', 'vinrace',


```
'willoughby', 'window', 'yellow']  
time: 39.6 ms (started: 2025-04-20 20:55:13 +02:00)
```

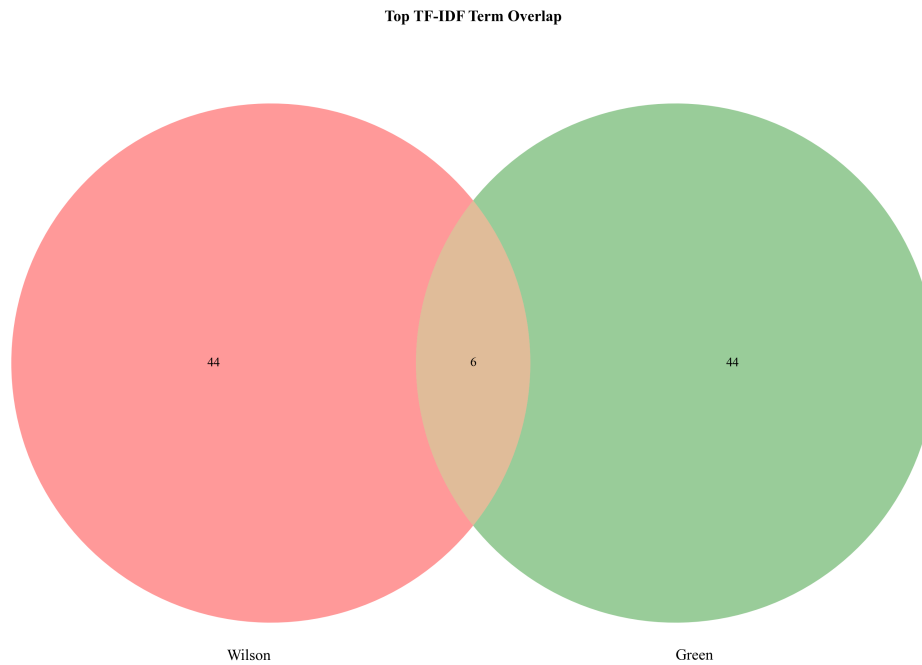
```
[61]: from matplotlib_venn import venn2, venn3  
  
# Example: Wilson vs Fagles  
venn2([top_terms_dict['Wilson'], top_terms_dict['Fagles']],  
      set_labels=('Wilson', 'Fagles'))  
plt.title("Top TF-IDF Term Overlap")  
plt.savefig(output_path_plots + f"venn_Wilson_Fagles-{nb_id}.png", dpi=400)  
plt.show()
```



```
time: 669 ms (started: 2025-04-20 20:55:13 +02:00)
```

```
[62]: #from matplotlib_venn import venn2, venn3  
  
# Example: Wilson vs Fagles  
venn2([top_terms_dict['Wilson'], top_terms_dict['Green']],  
      set_labels=('Wilson', 'Green'))  
plt.title("Top TF-IDF Term Overlap")
```

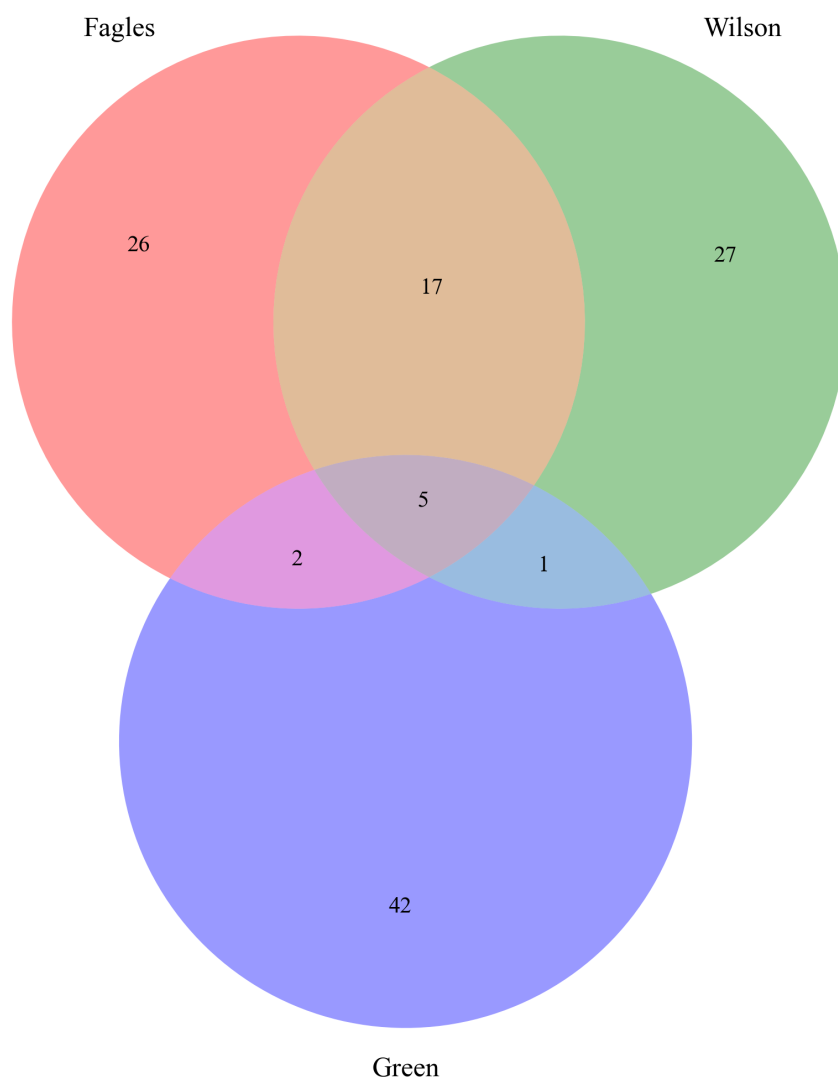
```
plt.savefig(output_path_plots + f"venn_Wilson_Green-{nb_id}.png", dpi=400)
plt.show()
```



time: 774 ms (started: 2025-04-20 20:55:14 +02:00)

```
[63]: venn3([top_terms_dict['Fagles'],top_terms_dict['Wilson'],  
            ↪top_terms_dict['Green']],  
            set_labels=('Fagles','Wilson', 'Green'))  
plt.title("Top TF-IDF Term Overlap")  
plt.savefig(output_path_plots + f"venn_Fagles_Wilson_Green-{nb_id}.png",  
            ↪dpi=400)  
plt.show()
```

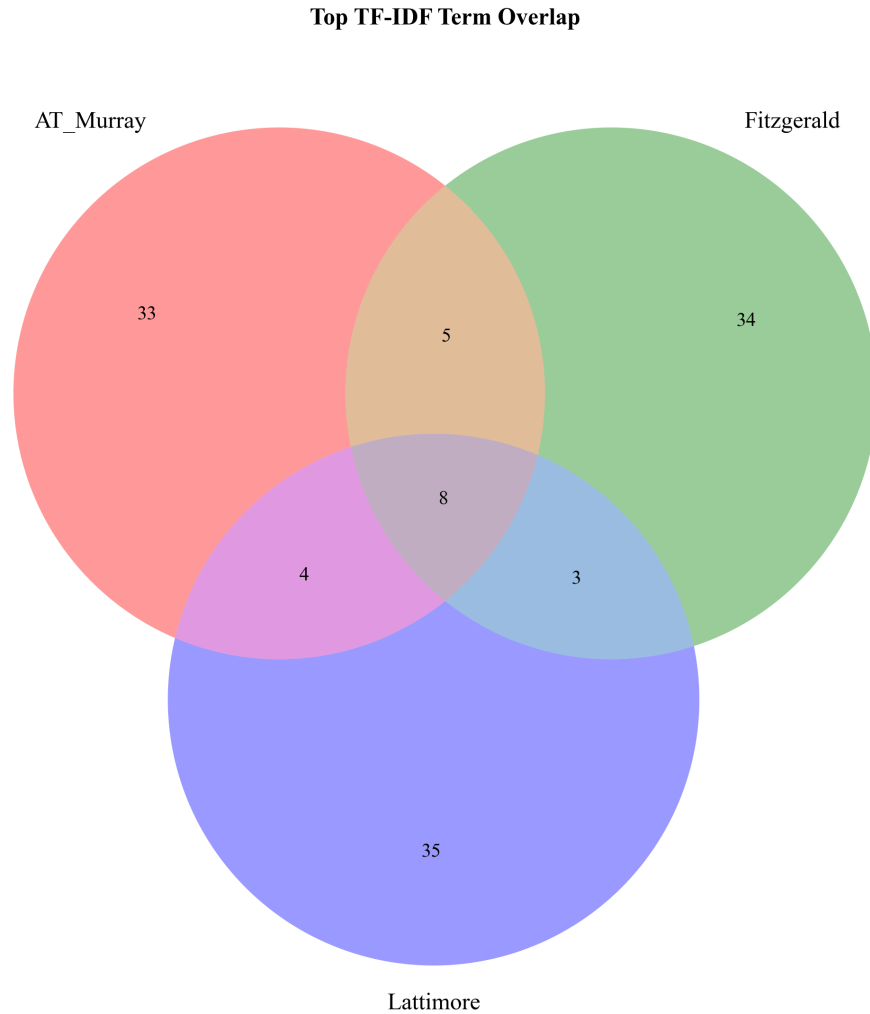
Top TF-IDF Term Overlap



time: 519 ms (started: 2025-04-20 20:55:14 +02:00)

```
[64]: venn3([top_terms_dict['AT_Murray'],top_terms_dict['Fitzgerald'],  
            ↪top_terms_dict['Lattimore']],  
           set_labels=('AT_Murray','Fitzgerald', 'Lattimore'))  
plt.title("Top TF-IDF Term Overlap")
```

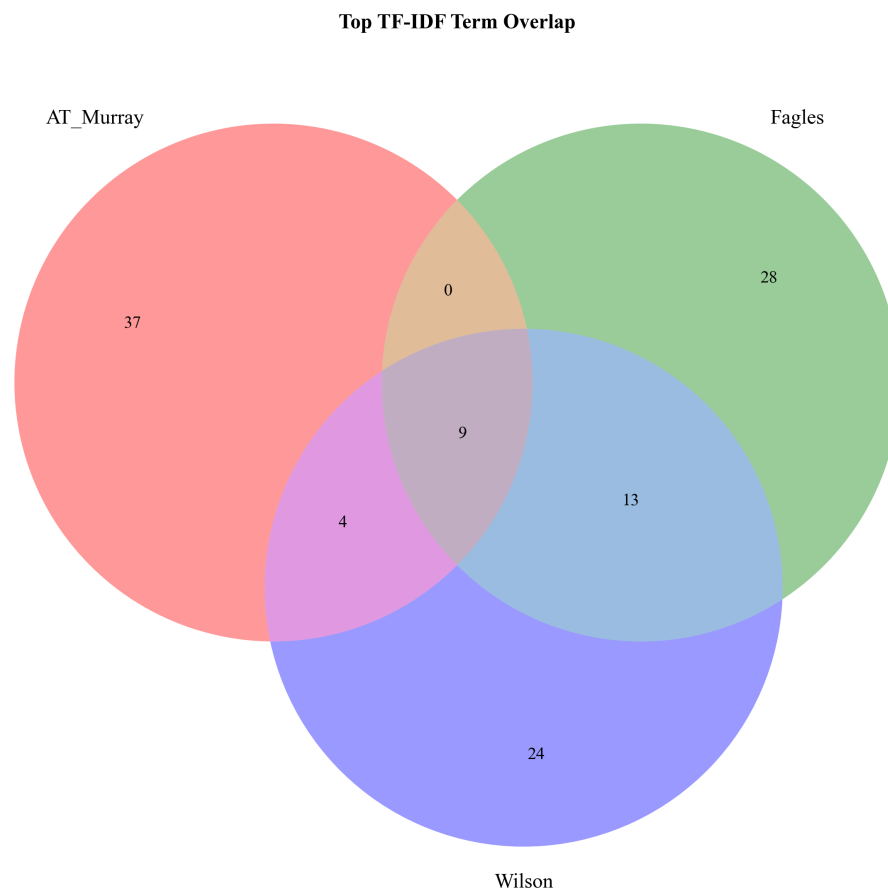
```
plt.savefig(output_path_plots + f"venn_AT_Murray_Fitzgerald_Lattimore-{nb_id}."
            ↪png", dpi=400)
plt.show()
```



time: 530 ms (started: 2025-04-20 20:55:15 +02:00)

```
[67]: venn3([top_terms_dict['AT_Murray'], top_terms_dict['Fagles'],
            ↪top_terms_dict['Wilson']],
            set_labels=('AT_Murray', 'Fagles', 'Wilson'))
plt.title("Top TF-IDF Term Overlap")
#plt.savefig(output_path_plots + f"venn_AT_Murray_Fitzgerald_Lattimore-{nb_id}."
            ↪png", dpi=400)
```

```
plt.show()
```



```
[65]: %unload_ext autotime
```

1.7 VI. Discussing the Results